

Project APEX Engineering Apprenticeship

Hands-on world models, state-space models, F1 telemetry and production simulation engineering

OpenAI

August 2026

Table of Contents

How to use this apprenticeship

This is not a catalogue of topics. Every chapter begins with a failure or engineering need, makes you predict an outcome, builds the smallest working system, inspects its internal state, breaks it deliberately, repairs it with a regression test, compares alternatives, and then transfers the idea into Project APEX.

Use a split screen: the book on one side and the repository on the other. Type the code. Do not merely read it. Before every output, write down what you expect. When your expectation is wrong, that gap is the lesson.

The teaching loop

Problem → Prediction → Small build → Visible trace → Deliberate failure → Diagnosis → Repair → Comparison → APEX integration → Independent challenge

Repository map

labs/ contains small isolated experiments.

projects/01...09/ contains progressively larger builds.

apex_engine/ is the production-structured capstone.

debugging_cases/ contains failures that should be investigated before reading the explanation.

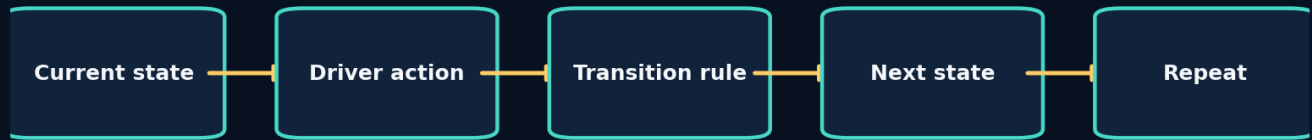
source_code_companion/ contains line-numbered explanations of the production source.

field_workbook/ is where you record predictions, experiment plans, and architecture decisions.

1. Build a Simulator Before You Build a Neural Network

Instructor objective: Understand simulation as repeated state transition, then build and inspect the smallest causal driving world.

A simulator is a transition repeated in time



The first world model can be a hand-written equation. Learning comes later.

1. Build a Simulator Before You Build a Neural Network

The problem that earns this chapter

You want a model that can imagine an F1 car several seconds into the future. It is tempting to open PyTorch immediately. That would hide the central question: **what does it mean for the world to move forward by one step?** Before learning anything from data, we need a transparent transition rule whose mistakes we can see.

Predict before reading

A car is moving at 20 m/s. During the next 0.2 seconds, its acceleration is 3 m/s². Predict the next velocity and the distance travelled if we use the simplest Euler update. Then decide which quantity should be updated first.

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

A simulator stores a description of the world, receives an intervention, applies a transition, and returns a new description. In our first world, the state is only velocity and position. The action is a throttle value. The transition is an equation. Repeating the equation produces a rollout.

This hand-written simulator is already a world model in the broad engineering sense: it predicts a future state from a present state and an action. It is not a *learned* world model. That difference matters. A learned model replaces part of the transition rule with parameters estimated from evidence. The interface should remain stable: state in, action in, next state out.

Starting here gives us an oracle for later debugging. If a neural network predicts that stronger braking increases velocity, we can compare it with the toy causal rule and ask whether the data contract, target alignment, or learned relationship is broken.

Vocabulary that now has a job

Concept: State - Meaning in plain language: The minimum information carried from one step to the next. - **Role inside APEX:** Speed, progress, tyre state and other dynamic variables.

Concept: Action - Meaning in plain language: A controllable input applied during a transition. - **Role inside APEX:** Throttle, brake, steering and gear choices.

Concept: Transition - Meaning in plain language: The rule that maps current state and action to next state. - **Role inside APEX:** Hand-written physics, a GRU, an SSM or an RSSM.

Concept: Rollout - Meaning in plain language: Repeatedly applying the transition to create a future trajectory. - **Role inside APEX:** Imagining several seconds of future telemetry.

Worked example: calculate it by hand

Euler integration uses

$$v_{t+1} = v_t + a_t \Delta t$$

and

$$x_{t+1} = x_t + v_t \Delta t.$$

With $v_t = 20$, $a_t = 3$, and $\Delta t = 0.2$:

Velocity change: $3 \times 0.2 = 0.6$ m/s.

Next velocity: $20 + 0.6 = 20.6$ m/s.

Distance using the current velocity: $20 \times 0.2 = 4.0$ m.

Next position, if starting at zero: 4.0 m.

A semi-implicit variant updates velocity first and then position, producing $20.6 \times 0.2 = 4.12$ m. Neither is universally “the answer”; the numerical method is an implementation choice whose error shrinks with smaller time steps.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/02_vectorized_kinematics

What we are about to build

A tiny vectorized integrator that converts an acceleration sequence into velocity and position. You will see that a rollout is just a loop with state carried between iterations.

Runnable implementation

```
import numpy as np
```

```
dt = 0.2
```

```
time = np.arange(0, 5, dt)
```

```
speed = 20 + 4*np.sin(time)
```

```
accel = np.gradient(speed, dt)
```

```
reconstructed = speed[0] + np.cumsum(accel)*dt
```

```
print("first accelerations:", accel[:5].round(3))
```

```
print("max reconstruction error:", float(np.max(np.abs(reconstructed-speed))))
```

Observed output from the packaged solution

```
first accelerations: [3.973 3.894 3.66 3.279 2.768]
```

```
max reconstruction error: 0.7946773231802453
```

Read the important lines like English

Code: `dt = 0.1` - **What the line is doing:** Define how much simulated time one transition represents. -

What to inspect: Changing `dt` changes both numerical error and sequence length.

Code: `velocity[i] = velocity[i-1] + acceleration[i-1] * dt` - **What the line is doing:** Apply one causal state transition. - **What to inspect:** The action at step $i-1$ affects state i , not state $i-1$.

Code: `position[i] = position[i-1] + velocity[i-1] * dt` - **What the line is doing:** Integrate velocity into position. - **What to inspect:** This implementation uses explicit Euler.

State and tensor trace

Trace one step with `velocity[0]=10`, `acceleration[0]=2`, and `dt=0.1`:

before: `velocity = 10.0 m/s`, `position = 0.0 m`

action: `acceleration = 2.0 m/s2`

change: $\Delta v = 2.0 \times 0.1 = 0.2 \text{ m/s}$

after: `velocity = 10.2 m/s`

position change: $10.0 \times 0.1 = 1.0 \text{ m}$

after: `position = 1.0 m`

The array is merely a stored history of repeated state updates.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Change `dt` from 0.1 to 1.0 without changing the acceleration profile. Compare the trajectory. Then update position with the newly computed velocity and compare again.

Diagnose from the earliest failed contract

If the two trajectories disagree, the code is not necessarily broken. The earliest changed contract is the numerical integration scheme and time resolution. Ask whether your result converges as `dt` becomes smaller.

Repair and lock the repair with a test

Add a test that simulates constant acceleration for one second at several time steps. Assert that final velocity approaches the analytical value $v_0 + a \cdot t$. Document the position integrator you chose.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Explicit Euler - Choose it when: You need the simplest transparent baseline and small time steps. - **Do not choose it when:** Long unstable rollouts or stiff dynamics.

Implementation: Semi-implicit Euler - Choose it when: Velocity drives position and you want slightly better stability at similar cost. - **Do not choose it when:** You need high-order accuracy.

Implementation: Runge–Kutta - Choose it when: The hand-written dynamics are smooth and numerical accuracy matters. - **Do not choose it when:** The transition is learned from discrete telemetry where solver precision is not the main bottleneck.

Implementation: Learned transition - Choose it when: Important dynamics are unknown but repeated observations are available. - **Do not choose it when:** You have not defined state, action, timing or evaluation.

For APEX, the toy simulator uses simple integration because its purpose is causal education and synthetic data generation. The learned model then predicts transitions on a fixed telemetry grid. We do not pretend the neural network is a high-fidelity rigid-body solver.

Transfer the lesson into Project APEX

The production engine keeps the same conceptual interface: a history encodes the current world, future controls supply actions, and the model emits future states. The complexity grows, but the causal contract learned here should remain visible.

Repository path to inspect

```
projects/01_car_integrator/main.py
apex_engine/src/apexsim/data/synthetic.py
apex_engine/src/apexsim/simulation.py
```

Connection to research

Dreamer-style systems are sophisticated because they learn transition dynamics in a latent state, but they still depend on this repeated transition idea. If you cannot explain the toy rollout, an RSSM will only hide confusion behind distributions.

Check your understanding before continuing

Why is a one-step transition different from a rollout?

What changes when dt is halved?

Why should actions at time t usually predict state at time $t+1$ rather than state at time t ?

Solutions and reasoning

1. A one-step transition consumes one state/action pair; a rollout feeds predictions back repeatedly, so errors change later inputs. 2. There are twice as many transitions for the same physical duration, usually reducing integration error but increasing compute and sequence length. 3. The state at time t was observed before or at the moment the action is applied; using the same-time target can leak the answer or teach no causal delay.

Independent build challenge

Implement both explicit and semi-implicit Euler. Plot their position error against the analytical solution for five time steps. Write an ADR choosing one for the synthetic APEX generator.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

2. State, Action, Context and the Markov Question

Instructor objective: Design a causal telemetry schema and discover when the visible state is insufficient for predicting the future.

Separate what the world is, what you do, and what you cannot control



Speed is state; throttle is action; rain is context.

2. State, Action, Context and the Markov Question

The problem that earns this chapter

Suppose two cars have the same speed and throttle. One is on fresh soft tyres in dry weather; the other is on worn hards in rain. A model that sees only speed and throttle receives identical input for futures that should differ. The failure is not “the neural network needs more layers.” The state description is incomplete.

Predict before reading

Classify each variable as state, action, context, target, or identifier: speed, throttle, rain intensity, driver number, tyre age, next speed, track name, brake. Explain any variable whose category could change depending on the system boundary.

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

State is the information the transition needs to carry. Action is the intervention being chosen. Context changes the transition but is not controlled by the agent inside the model. Identifiers describe an entity or grouping and should not automatically become numeric features.

The Markov property says the next state is conditionally independent of the distant past once the present state and action are known. Real telemetry is rarely perfectly Markov. Tyre temperature, fuel mass, setup, and previous cornering load may be hidden. A recurrent model compensates by compressing history into memory. But recurrence is not permission to ignore schema design: hidden memory cannot recover information that was never observed.

A practical state contract should be sufficient, measurable, stable across data sources, and safe to use at inference time. A variable known only after the race cannot be an input to an online simulator.

Vocabulary that now has a job

Concept: Markov state - Meaning in plain language: A state that contains enough information to predict the next transition when combined with the action. - **Role inside APEX:** The ideal telemetry state; approximated using features plus history.

Concept: Context - Meaning in plain language: An external condition that modifies dynamics. - **Role inside APEX:** Rain, track geometry, session phase and compound.

Concept: Identifier - Meaning in plain language: A label used for grouping or lookup, not automatically a causal feature. - **Role inside APEX:** Session ID, driver code and event ID.

Concept: Partial observability - Meaning in plain language: Important state exists but is not directly measured. - **Role inside APEX:** Tyre temperature or setup may need history, proxies or latent state.

Worked example: calculate it by hand

Consider the transition

$$v_{t+1} = v_t + (4a_t - 7b_t - 0.02v_t - 3r_t)\Delta t$$

where a is throttle, b is brake, and r is rain intensity. At $v=50$ m/s, throttle 0.8, brake 0, rain 0.5, and $dt=0.2$:

Engine contribution: $4 \times 0.8 = 3.2$ m/s².

Drag contribution: $0.02 \times 50 = 1.0$ m/s².

Rain/grip penalty: $3 \times 0.5 = 1.5$ m/s².

Net acceleration: $3.2 - 1.0 - 1.5 = 0.7$ m/s².

Speed change: $0.7 \times 0.2 = 0.14$ m/s.

Next speed: 50.14 m/s.

If rain were omitted, the same visible state/action would predict 50.44 m/s. That 0.30 m/s discrepancy is a schema problem before it is a modelling problem.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/01_units_and_shapes

What we are about to build

A two-feature telemetry tensor with explicit time and feature axes. The point is not the arithmetic; it is learning to make shape and semantics part of the data contract.

Runnable implementation

```
import numpy as np
```

```
speed_kmh = np.array([0.0, 72.0, 144.0])
speed_mps = speed_kmh / 3.6
telemetry = np.stack([speed_mps, np.array([0.0, 0.5, 1.0])], axis=1)
print("speed_mps:", speed_mps)
print("telemetry shape [time, features]:", telemetry.shape)
assert telemetry.shape == (3, 2)
```

Observed output from the packaged solution

```
speed_mps: [ 0. 20. 40.]
telemetry shape [time, features]: (3, 2)
```

Read the important lines like English

Code: `speed_mps = speed_kmh / 3.6` - **What the line is doing:** Convert a display unit into the canonical physical unit. - **What to inspect:** The numerical values change while the physical quantity does not.

Code: `np.stack(..., axis=1)` - **What the line is doing:** Create one row per time step and one column per feature. - **What to inspect:** `axis=1` establishes [time, features].

Code: `assert telemetry.shape == (3, 2)` - **What the line is doing:** Turn the axis contract into an executable check. - **What to inspect:** A shape check cannot verify feature order or units by itself.

State and tensor trace

```
row 0 = [0.0 m/s, 0.0 throttle]
row 1 = [20.0 m/s, 0.5 throttle]
row 2 = [40.0 m/s, 1.0 throttle]
shape = [3 time steps, 2 features]
```

If you transpose it, the shape becomes [2 features, 3 time steps]; the numbers still exist, but every downstream interpretation changes.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Transpose the array and remove the shape assertion. Write a function that assumes rows are time. Observe that it can compute plausible but meaningless statistics.

Diagnose from the earliest failed contract

The earliest failed contract is axis semantics, not the model. Print shape, feature names, units, and one known row before debugging anything downstream.

Repair and lock the repair with a test

Create a dataclass or Pydantic model that stores `feature_names`, `units`, `sample_hz`, and the array. Add tests for feature order, legal ranges, and axis length.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Flat numeric vector - **Choose it when:** The state is small, fixed and well documented. - **Do not choose it when:** Variables have complex structure or missingness semantics.

Implementation: Typed record / dataframe - **Choose it when:** You are ingesting and validating human-readable telemetry. - **Do not choose it when:** You need high-throughput batched neural computation.

Implementation: History encoder - **Choose it when:** The observed frame is partially Markov and recent history contains useful proxies. - **Do not choose it when:** Missing variables are unrelated to anything observed.

Implementation: Latent state model - **Choose it when:** Hidden dynamics and uncertainty must be inferred. - **Do not choose it when:** A transparent state vector already predicts well enough.

APEX uses named tabular records at ingestion, then ordered tensors for learning. Conversion happens at an explicit boundary so readability and computational efficiency are both preserved.

Transfer the lesson into Project APEX

The canonical contract distinguishes state features, control features, context features and identifiers. The window builder constructs history and future-control tensors without exposing future target state.

Repository path to inspect

```
apex_engine/src/apexsim/contracts.py
apex_engine/src/apexsim/data/features.py
apex_engine/src/apexsim/data/windows.py
```

Connection to research

RSSMs exist partly because observations are incomplete. Their deterministic hidden state and stochastic latent state summarize what the agent believes about the world. That belief is only as good as the evidence and intervention contract supplied to it.

Check your understanding before continuing

Can tyre age be state and context at the same time?

Why is driver ID dangerous as a direct numeric feature?

What evidence would show that a single telemetry frame is not Markov enough?

Solutions and reasoning

1. The label depends on the chosen boundary. If tyre age evolves inside the simulator, it is state; if supplied as a fixed scenario condition, it behaves like context. 2. Numeric encoding invents an ordering and can let the model memorize identities instead of dynamics. Use a justified embedding or group-specific analysis. 3. A history-aware model consistently outperforming an equal-capacity frame model, especially after controlling leakage, suggests the past carries missing state information.

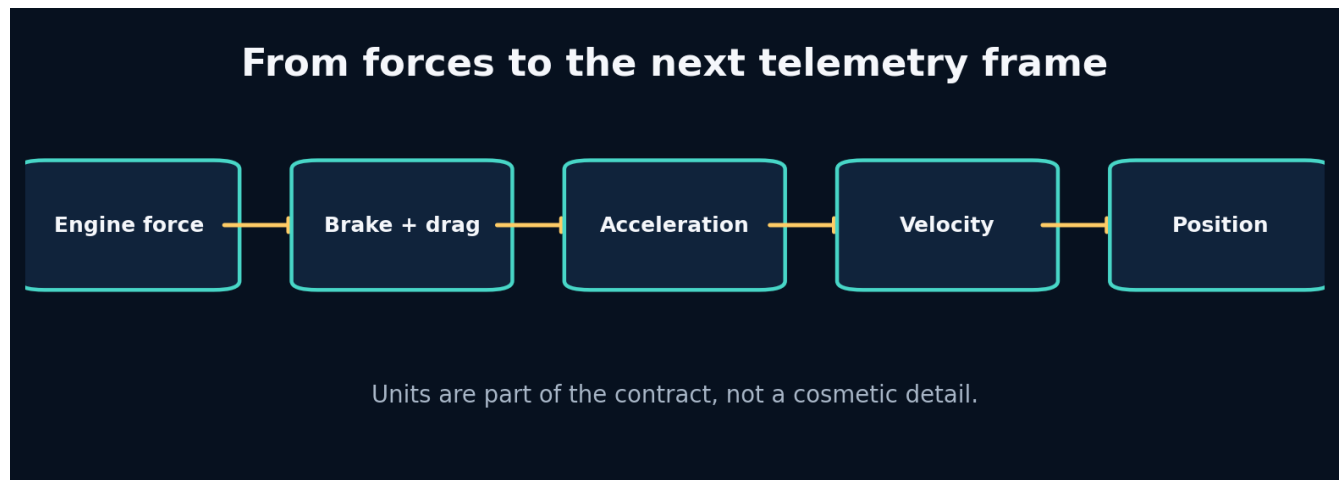
Independent build challenge

Write the complete APEX V1 schema on paper. For every field, state its category, unit, legal range, availability at inference, and expected causal relationship.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

3. Units, Forces and Numerical Integration

Instructor objective: Build a physically interpretable transition, use dimensional analysis to catch errors, and understand the limits of toy physics.



3. Units, Forces and Numerical Integration

The problem that earns this chapter

Telemetry sources use kilometres per hour, metres per second, percentages, normalized controls, degrees, radians and timestamps in different formats. A model can fit unit mistakes without raising an exception. The result may look statistically smooth while violating the world.

Predict before reading

A formula subtracts $0.02 * \text{speed}^2$ from acceleration, but speed is supplied in km/h although the coefficient was fitted in m/s. Predict whether the drag is too large or too small and by what factor.

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

Dimensional analysis asks whether the units on both sides of an equation agree. It is one of the cheapest and strongest debugging tools in simulation engineering. If force is in newtons and mass is in kilograms, acceleration is metres per second squared. If speed enters a squared term, a unit conversion error is squared too.

A toy longitudinal model combines engine force, braking force and drag. It will not reproduce downforce, tyre load sensitivity, energy recovery, differential behaviour or setup effects. That is fine when the model's purpose is to generate causal examples and test software contracts. It is dangerous if the UI presents the result as lap-time truth.

The hand-written model also teaches residual learning: instead of asking a neural network to rediscover every obvious relationship, we can predict the correction to a known physics approximation. Whether that helps is an empirical decision.

Vocabulary that now has a job

Concept: Dimensional analysis - Meaning in plain language: Check that quantities combined by an equation have compatible units. - **Role inside APEX:** Catches km/h–m/s and timestamp errors before training.

Concept: Force balance - Meaning in plain language: Net force is the sum of propulsive and resisting forces. - **Role inside APEX:** Provides an interpretable synthetic transition.

Concept: Residual model - Meaning in plain language: A learned correction added to a known baseline model. - **Role inside APEX:** Possible hybrid physics/ML architecture.

Concept: Numerical stability - Meaning in plain language: Whether repeated computation remains bounded and sensible. - **Role inside APEX:** Critical for long autoregressive rollouts and SSM state.

Worked example: calculate it by hand

Suppose the drag acceleration term is $c v^2$. Converting from m/s to km/h multiplies speed by 3.6. Squaring multiplies the drag term by $3.6^2 = 12.96$. The model therefore applies nearly thirteen times too much drag.

For a 50 m/s car with mass 800 kg:

Engine force: 8,000 N

Brake force: 0 N

Drag: 2,500 N

Net force: 5,500 N

Acceleration: $5500 / 800 = 6.875 \text{ m/s}^2$

At $dt=0.1$, velocity increases by 0.6875 m/s

If speed were incorrectly treated as 180 in a coefficient calibrated for 50, the quadratic term would dominate and could create impossible negative velocity.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/03_force_model

What we are about to build

A longitudinal force model with engine, brake and aerodynamic drag contributions. You will inspect every term separately before summing them.

Runnable implementation

from education.core **import** CarState, step_car

state = CarState(0.0, 0.0, 40.0)

for i **in** range(5):

state = step_car(state, throttle=0.7, brake=0.0, dt=0.1)

print(i, state)

Observed output from the packaged solution

Traceback (most recent call last):

File "/mnt/data/Project_APEX_Engineering_Apprenticeship/labs/03_force_model/solution.py", line 1, in <module>

from education.core import CarState, step_car

ModuleNotFoundError: No module named 'education'

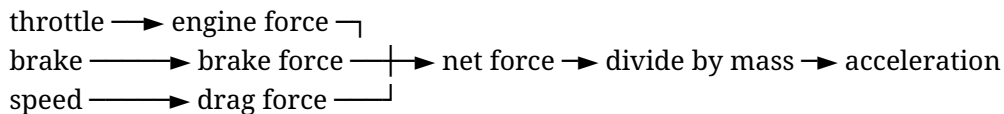
Read the important lines like English

Code: `engine = max_engine_force * throttle` - **What the line is doing:** Scale available forward force by a normalized control. - **What to inspect:** Throttle range must be validated as 0–1.

Code: `drag = drag_coefficient * speed2` - **What the line is doing:** Apply a quadratic resistance that grows rapidly with speed. - **What to inspect:** The coefficient depends on the speed unit.

Code: `acceleration = net_force / mass` - **What the line is doing:** Use Newton's second law to convert force into acceleration. - **What to inspect:** Mass must be positive and use kilograms.

State and tensor trace



At every arrow, annotate the unit. That annotation is an executable design review even before a test exists.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Feed speed in km/h without conversion. Then set mass to zero or throttle above one. Observe which failures become exceptions and which silently produce nonsense.

Diagnose from the earliest failed contract

Start with invariants: $mass > 0$, $0 \leq controls \leq 1$, speed unit known, $drag \geq 0$, and braking should not increase velocity. Then inspect individual force terms.

Repair and lock the repair with a test

Introduce typed conversion functions, range validation, and property tests: increasing brake at fixed state must not increase next speed; increasing throttle must not decrease it in a regime without traction limits.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Pure hand-written physics - Choose it when: You need interpretability, counterfactual control and known equations. - **Do not choose it when:** Unknown tyre/track interactions dominate error.

Implementation: Pure learned dynamics - Choose it when: You have broad representative data and care about predictive fit. - **Do not choose it when:** Safety or extrapolation requires hard physical guarantees.

Implementation: Residual learning - Choose it when: A baseline captures broad physics and data can learn systematic corrections. - **Do not choose it when:** The baseline is badly misspecified and constrains learning in the wrong direction.

Implementation: Physics-informed loss - Choose it when: Violations can be expressed as differentiable penalties. - **Do not choose it when:** The penalty is only a weak proxy and overwhelms observed evidence.

APEX V1 uses the toy force model to create controllable offline data, while the predictive models learn directly from canonical telemetry. Physical validity checks remain part of evaluation rather than being assumed from the architecture.

Transfer the lesson into Project APEX

Inspect how the synthetic generator computes curvature demand, grip, drag, controls and state updates. Then compare those equations with the features available to the learned models.

Repository path to inspect

```
projects/03_lap_physics/main.py
apex_engine/src/apexsim/data/synthetic.py
apex_engine/src/apexsim/evaluation.py
```

Connection to research

World models often optimize data likelihood or prediction error, not the laws of mechanics. Adding inductive bias can improve sample efficiency, but every constraint is a claim that must be tested under the target domain.

Check your understanding before continuing

Why can two unit errors cancel during training but still fail in deployment?

What is the advantage of predicting a residual?

Name one invariant that should hold for all APEX rollouts.

Solutions and reasoning

1. A model can learn coefficients adapted to the mistaken training scale; a different source or UI conversion changes the scale and exposes the mismatch. 2. The baseline supplies known structure, so the learner spends capacity on unmodelled effects and can be easier to interpret. 3. Examples include finite values, nonnegative speed, track progress wrapped into its legal interval, and controls within their defined range.

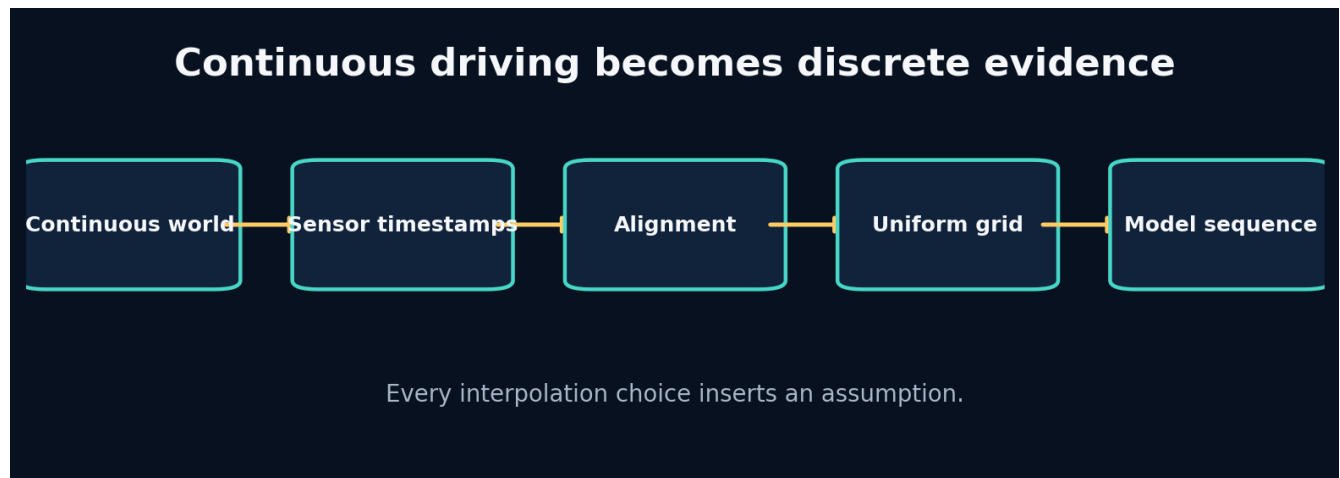
Independent build challenge

Add a tyre-grip limit to the force model. Show a case where more throttle no longer produces more acceleration, and explain why a monotonicity test must be conditional rather than universal.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

4. Sampling, Aliasing and Time Alignment

Instructor objective: Turn asynchronous telemetry into a defensible time grid without creating fake evidence or future leakage.



4. Sampling, Aliasing and Time Alignment

The problem that earns this chapter

Speed may arrive at one frequency, weather at another, and event data only when something changes. A neural network expects aligned rows. Joining by nearest timestamp can quietly attach a future weather measurement to an earlier car frame. Interpolation can make missing data look observed.

Predict before reading

An 8 Hz oscillation is sampled at 5 Hz. Sketch the samples. Will the observed sequence reveal an 8 Hz signal, a slower false signal, or no signal? Then decide what a model might learn from it.

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

Sampling converts a continuous process into discrete observations. If the sample rate is too low for the dynamics, high-frequency behaviour aliases into a different pattern. No architecture can reconstruct information that the sensor never captured without additional assumptions.

Alignment chooses which measurements describe the same model time. A backward as-of join says: at each car timestamp, use the latest context that was already available. A nearest join may use the future. A tolerance limits how stale a matched measurement may be. These are causal policies, not dataframe trivia.

Resampling onto a uniform grid simplifies windows and batch training, but interpolation must respect feature type. Continuous speed may be linearly interpolated across short gaps. Gear and flags are categorical and generally require forward fill or explicit unknown values. Long gaps should be marked invalid, not painted over.

Vocabulary that now has a job

Concept: Sampling rate - **Meaning in plain language:** How many observations are recorded per second. - **Role inside APEX:** Defines temporal resolution and sequence length.

Concept: Aliasing - **Meaning in plain language:** A high-frequency process appears as a different lower-frequency pattern. - **Role inside APEX:** Can hide braking spikes or steering oscillations.

Concept: As-of join - Meaning in plain language: Match each timestamp with a nearby record under a direction and tolerance policy. - **Role inside APEX:** Aligns weather, position and telemetry streams.

Concept: Staleness - Meaning in plain language: How old a context value is when reused. - **Role inside APEX:** A monitoring feature and validation rule.

Worked example: calculate it by hand

The Nyquist guideline says a sinusoid of frequency f requires sampling above $2f$ to avoid ambiguity. An 8 Hz oscillation needs more than 16 Hz. At 5 Hz, sample times are 0, 0.2, 0.4... seconds. The phase advances $8 \times 0.2 = 1.6$ cycles per sample, equivalent to an apparent 0.6-cycle advance after wrapping. The observed pattern therefore resembles a 3 Hz oscillation (0.6×5).

For alignment, suppose car timestamps are 0.0, 0.2, 0.4, 0.6 and rain observations are 0.05 and 0.55. A backward join at 0.4 uses rain from 0.05; a nearest join uses 0.55, leaking a future value by 0.15 seconds.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/19_time_alignment

What we are about to build

A backward merge_asof with an explicit tolerance. You will inspect which rain observation is attached to each car frame and identify unmatched stale rows.

Runnable implementation

import pandas **as** pd

```
car=pd.DataFrame({'t':[0.0,0.2,0.4,0.6],'speed':[10,12,14,16]})
weather=pd.DataFrame({'t':[0.05,0.55],'rain':[0.0,0.3]})
aligned=pd.merge_asof(car.sort_values('t'),weather.sort_values('t'),on='t',direction='backward',tolerance=0.3)
print(aligned)
```

Observed output from the packaged solution

```
t speed rain
0 0.0  10 NaN
1 0.2  12  0.0
2 0.4  14 NaN
3 0.6  16  0.3
```

Read the important lines like English

Code: sort_values('t') - **What the line is doing:** Satisfy the ordered-time precondition of an as-of join. - **What to inspect:** Unsorted data can fail or produce invalid matches.

Code: direction='backward' - **What the line is doing:** Use only context observations at or before the car timestamp. - **What to inspect:** This encodes online availability and blocks future leakage.

Code: tolerance=0.3 - **What the line is doing:** Reject a match older than the allowed staleness. - **What to inspect:** Choose tolerance from domain timing, not convenience.

State and tensor trace

car t=0.00 → no earlier rain measurement → missing
 car t=0.20 → rain at 0.05 (age 0.15 s) → valid
 car t=0.40 → rain at 0.05 (age 0.35 s) → rejected by 0.30 s tolerance
 car t=0.60 → rain at 0.55 (age 0.05 s) → valid

A table row should carry not only the matched value but ideally the age of that value.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Change direction to nearest and remove the tolerance. Add a future rain spike at 0.41. Observe how an earlier car frame receives information from the future.

Diagnose from the earliest failed contract

Compare source timestamp, target timestamp and signed age. The earliest failed contract is availability time, not model performance.

Repair and lock the repair with a test

Persist measurement timestamps and create a test asserting `context_time <= frame_time` for online features. Add a maximum staleness threshold and an explicit missingness mask.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Backward as-of - Choose it when: Features must reflect information available online.
- Do not choose it when: The quantity is defined as a symmetric offline estimate.

Implementation: Nearest join - Choose it when: Both streams are synchronized measurements and future use is scientifically acceptable. **- Do not choose it when:** Forecasting, control or causal analysis.

Implementation: Linear interpolation - Choose it when: A continuous signal has short, well-sampled gaps. **- Do not choose it when:** Categorical states, discontinuities or long outages.

Implementation: Forward fill - Choose it when: A discrete state remains active until changed. **- Do not choose it when:** A rapidly varying continuous sensor.

APEX chooses a canonical frequency, aligns source streams under causal policies, records staleness, and rejects gaps that exceed configured limits. The policy is part of the dataset version.

Transfer the lesson into Project APEX

The FastF1 and OpenF1 adapters normalize timestamps and stream-specific fields before producing the canonical frame. Window construction assumes the alignment contract is already satisfied.

Repository path to inspect

labs/04_sampling_aliasing/solution.py
 labs/19_time_alignment/solution.py
 apex_engine/src/apexsim/data/fastf1_adapter.py
 apex_engine/src/apexsim/data/openf1_adapter.py

Connection to research

Sequence models are often blamed for missing rapid dynamics that were already destroyed by low-rate or misaligned observations. Data acquisition defines the ceiling of learnable temporal behaviour.

Check your understanding before continuing

Why does increasing model capacity not solve aliasing?

What does a join tolerance represent physically?

When can interpolation create false certainty?

Solutions and reasoning

1. The distinct high-frequency processes map to identical samples, so the information is absent from the data. 2. The maximum age or temporal mismatch at which a context measurement is still considered representative. 3. When it fills a long or discontinuous gap with smooth values and the training pipeline treats those values as directly observed.

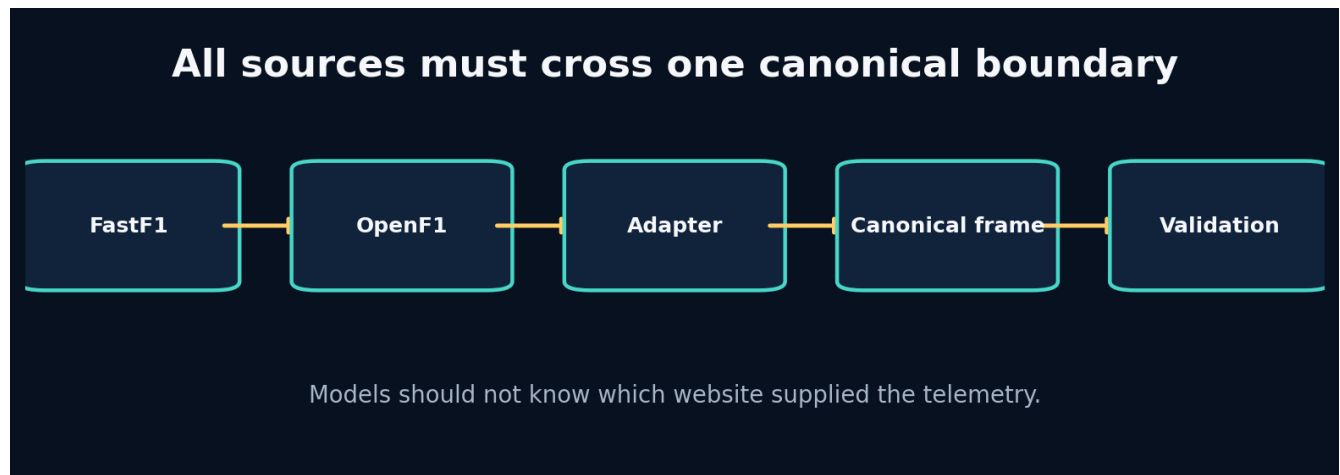
Independent build challenge

Create two asynchronous synthetic streams with a known causal delay. Compare nearest, backward and interpolated alignment by measuring how well each preserves the known delay.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

5. Canonical Contracts and Quality Gates

Instructor objective: Build one source-independent telemetry contract and make invalid evidence fail before it reaches a model.



5. Canonical Contracts and Quality Gates

The problem that earns this chapter

FastF1, OpenF1, synthetic data and future F1 25 UDP packets do not share identical names, units, timing or missingness. If every model contains source-specific branches, the system becomes impossible to test. We need a narrow boundary that all sources must cross.

Predict before reading

Imagine FastF1 supplies speed in km/h and a future UDP adapter supplies m/s. Both columns are called speed. Where should conversion happen: inside each model, inside the dataset, or inside the source adapter? Explain the consequences of each choice.

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

A canonical contract is not merely a dataframe schema. It specifies names, types, units, legal ranges, timing semantics, missingness, identifiers and invariants. Source adapters are translators: they may parse and convert, but they should not secretly perform model-specific feature engineering.

A quality gate separates admissible evidence from rejected or quarantined evidence. Some checks are hard errors—missing timestamp, impossible unit, duplicate key. Others are warnings—unusual prevalence or a short gap. The distinction should reflect whether downstream interpretation remains valid.

Contracts also enable replaceability. You can test the entire pipeline with synthetic data, then swap to FastF1 without changing the window builder or model. This is the central architectural move that lets APEX start before F1 25 is available.

Vocabulary that now has a job

Concept: Canonical contract - Meaning in plain language: One stable internal representation accepted by all downstream components. - **Role inside APEX:** The source-independent telemetry frame.

Concept: Adapter - Meaning in plain language: A boundary component translating one external source into the canonical representation. - **Role inside APEX:** FastF1, OpenF1 and future UDP adapters.

Concept: Quality gate - Meaning in plain language: Executable rules deciding whether evidence may proceed. - **Role inside APEX:** Range, timing, duplicate, finite-value and schema checks.

Concept: Quarantine - Meaning in plain language: Preserve questionable data separately instead of silently dropping or using it. - **Role inside APEX:** Allows investigation and reprocessing after a fix.

Worked example: calculate it by hand

Suppose the canonical speed unit is m/s and the legal range is 0–105 m/s. A source row says 320 km/h. Adapter recognizes source unit km/h.

Convert: $320 / 3.6 = 88.89$ m/s.

Validate finite: yes.

Validate range: yes.

Store canonical value with source provenance.

If conversion were delayed until model code, validation would see 320 and reject a valid record—or the range would be widened and allow truly impossible m/s values. Therefore conversion belongs at the adapter boundary.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/20_pipeline_contract

What we are about to build

A miniature pipeline that writes one immutable artifact per stage. Although small, it teaches the idea that every stage has named input/output and observable status.

Runnable implementation

```
from pathlib import Path
import json
```

```
run=Path('artifacts/education_pipeline'); run.mkdir(parents=True,exist_ok=True)
stages=['ingest','validate','window','train','evaluate','publish']
for i,name in enumerate(stages):
    path=run/f'{i:02d}_{name}.json'; path.write_text(json.dumps({'stage':name,'status':'succeeded'}))
    print(path)
```

Observed output from the packaged solution

```
artifacts/education_pipeline/00_ingest.json
artifacts/education_pipeline/01_validate.json
artifacts/education_pipeline/02_window.json
artifacts/education_pipeline/03_train.json
artifacts/education_pipeline/04_evaluate.json
artifacts/education_pipeline/05_publish.json
```

Read the important lines like English

Code: `stages=['ingest','validate','window','train','evaluate','publish']` - **What the line is doing:**

Declare the lifecycle as distinct responsibilities. - **What to inspect:** A stage name should correspond to one recoverable contract.

Code: `run/f'{i:02d}_{name}.json'` - **What the line is doing:** Give each output a deterministic path within a run. - **What to inspect:** Production artifacts also need version and content metadata.

Code: `{'stage':name,'status':'succeeded'}` - **What the line is doing:** Persist machine-readable state instead of relying on console text. - **What to inspect:** Real status should be written atomically after success.

State and tensor trace

external row

↓ adapter parses names and units

canonical row

↓ validator checks invariants

admitted row + quality report

↓ window builder
model-ready example

Each arrow is a contract boundary with tests. A failure should name the earliest boundary it violates.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Rename one source column, duplicate a timestamp, insert infinite speed, and swap throttle/brake. Notice that only some defects are detectable from shape and range.

Diagnose from the earliest failed contract

Validate in layers: schema → types → units → temporal keys → ranges → cross-field relationships → distribution warnings. A semantic swap may require known examples or correlation checks.

Repair and lock the repair with a test

Create fixtures for one valid row and one row for every failure class. Make adapters produce the same canonical column order and metadata. Record source, adapter version and conversion policy.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Loose dataframe convention - Choose it when: Exploratory one-off analysis. - **Do not choose it when:** Multiple sources, teams or production reuse.

Implementation: Pydantic/dataclass contract - Choose it when: You need typed configuration and row/batch metadata validation. - **Do not choose it when:** Per-element validation becomes a throughput bottleneck; validate batches at boundaries.

Implementation: Schema registry - Choose it when: Many producers and consumers evolve independently. - **Do not choose it when:** A small local project where the operational overhead exceeds value.

Implementation: Silent coercion - Choose it when: Almost never; only for explicitly documented benign parsing. - **Do not choose it when:** Any conversion that can change scientific meaning.

APEX adapters own source parsing and canonical conversion. Core training code imports only the canonical contract. Validation produces a report and blocks hard failures before sequence construction.

Transfer the lesson into Project APEX

Read the contract and both adapters side by side. Confirm that training, evaluation and UI modules never call FastF1 or OpenF1 directly.

Repository path to inspect

```
apex_engine/src/apexsim/contracts.py
apex_engine/src/apexsim/data/validate.py
apex_engine/src/apexsim/data/fastf1_adapter.py
apex_engine/src/apexsim/data/openf1_adapter.py
```

Connection to research

Research code often assumes a fixed benchmark tensor. Production world models need a stronger boundary because source evolution can alter semantics without changing array shape.

Check your understanding before continuing

Why should source adapters avoid model-specific feature engineering?

Give an example of a warning rather than a hard error.

What metadata is needed to reproduce a canonical dataset?

Solutions and reasoning

1. It couples data acquisition to one model and creates inconsistent versions across experiments; stable raw canonical features should precede reproducible transforms. 2. A session with unusually high rain or a slightly lower sample rate may be valid but deserves attention. 3. Source identifiers, retrieval time, adapter version, units, alignment policy, validation configuration, feature code version and dataset hash.

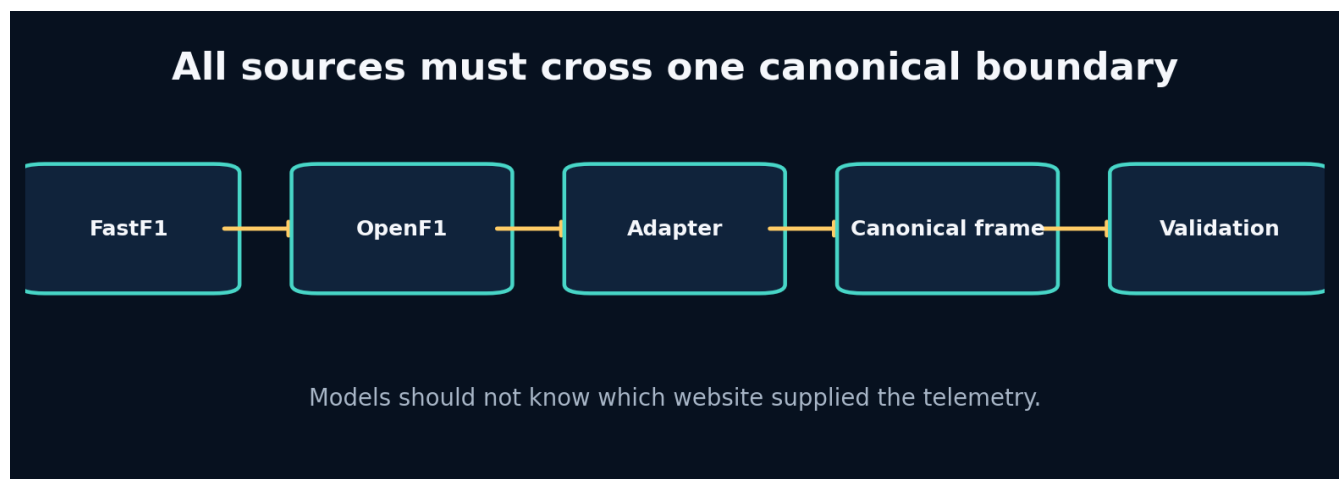
Independent build challenge

Define a canonical telemetry schema in JSON Schema or Pydantic. Implement two toy adapters with different names and units, and prove with tests that they produce identical canonical rows.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

6. FastF1 and OpenF1: Ingest Real Evidence Without Polluting the Core

Instructor objective: Understand the complete data path from a public F1 source to canonical telemetry and learn to keep network concerns outside model code.



6. FastF1 and OpenF1: Ingest Real Evidence Without Polluting the Core

The problem that earns this chapter

APEX begins before you own F1 25, so historical evidence must come from public tooling. FastF1 is a Python library organized around events and sessions; OpenF1 exposes HTTP endpoints for telemetry, timing, position, weather and related records. Their access patterns differ. The engineering challenge is not “download a CSV”; it is to translate each source into evidence with the same meaning.

Predict before reading

A FastF1 session and an OpenF1 endpoint both provide speed, throttle and timestamps, but one uses relative session time and the other UTC timestamps. What information must an adapter preserve so the two sources can later be compared or combined?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

Network retrieval, caching, parsing, source-specific naming and canonical conversion are separate steps. Keeping them separate lets you replay raw responses when an adapter changes, test conversion without the network, and avoid rate-limit failures inside training.

The safest ingestion process lands raw evidence first. A second deterministic step creates canonical data. This gives you two forensic layers: “what did the source return?” and “how did our code interpret it?” A model run should point to both.

FastF1 and OpenF1 can overlap without being interchangeable. Differences in sampling, provenance and field definitions should be measured rather than assumed away. APEX treats each adapter as an independently tested producer of the canonical contract.

Vocabulary that now has a job

Concept: Landing zone - Meaning in plain language: Immutable storage of source-native responses before interpretation. - **Role inside APEX:** Cached session/API data used for reproducible adapter tests.

Concept: Provenance - Meaning in plain language: Information describing where, when and how a record was obtained. - **Role inside APEX:** Source, event/session identifiers, driver, retrieval time and adapter version.

Concept: Idempotent ingestion - Meaning in plain language: Repeating the same request does not create conflicting outputs. - **Role inside APEX:** Deterministic paths and hashes for source snapshots.

Concept: Rate limit / cache - Meaning in plain language: External service constraints and local reuse strategy. - **Role inside APEX:** Prevents training from repeatedly hitting public services.

Worked example: calculate it by hand

Suppose OpenF1 returns UTC timestamps 12:00:00.100, 12:00:00.300, and FastF1 returns session-relative seconds 5.2, 5.4. To combine them, you need a session start reference or a shared event clock. If session start is 11:59:54.900, the UTC records map to 5.2 and 5.4 seconds.

The adapter should preserve:

Source timestamp exactly as received.

Parsed canonical timestamp.

Session and driver identifiers.

Source field names/units or adapter version.

Missingness and interpolation flags.

Without those fields, an apparent 0.2-second alignment error cannot be traced.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/19_time_alignment

What we are about to build

Use the alignment lab as an offline stand-in for real streams, then inspect the production FastF1 and OpenF1 adapters. The important skill is tracing a field from raw response to canonical column.

Runnable implementation

```
import pandas as pd
```

```
car=pd.DataFrame({'t':[0.0,0.2,0.4,0.6],'speed':[10,12,14,16]})
```

```
weather=pd.DataFrame({'t':[0.05,0.55],'rain':[0.0,0.3]})
```

```
aligned=pd.merge_asof(car.sort_values('t'),weather.sort_values('t'),on='t',direction='backward',tolerance=0.3)
```

```
print(aligned)
```

Observed output from the packaged solution

```
t speed rain
0 0.0  10 NaN
1 0.2  12  0.0
2 0.4  14 NaN
3 0.6  16  0.3
```

Read the important lines like English

Code: `pd.merge_asof(...)` - What the line is doing: Join asynchronous records after each source has been parsed into a common time representation. - **What to inspect:** Alignment belongs after timestamp normalization.

Code: `direction="backward"` - What the line is doing: Preserve online causal availability. - **What to inspect:** Offline retrospective analysis may choose another policy, but must label it.

Code: `tolerance=...` - What the line is doing: Prevent arbitrarily stale context from being treated as current. - **What to inspect:** Record unmatched rows rather than silently filling all gaps.

State and tensor trace

network request

↓ raw JSON / library objects (cache this)

source parser

↓ typed source table

unit and timestamp conversion

↓ canonical columns

quality gate

↓ admitted canonical session + report

Training begins only after the last step. It should be possible to run everything after ingestion with no network.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Mock a source response where one field changes name, one timestamp is timezone-naive, and one speed column changes unit. Run the adapter tests and identify which change is detected.

Diagnose from the earliest failed contract

Compare the raw snapshot with the canonical output. If the raw source is correct but canonical data is wrong, the adapter is the earliest failed boundary. If the raw response itself changed, update the source fixture and conversion policy deliberately.

Repair and lock the repair with a test

Store representative source fixtures in tests. Assert exact canonical names, units, timestamp monotonicity and a few known values. Version adapter outputs whenever semantics change.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: FastF1 - Choose it when: Python-centric historical session analysis with convenient session abstractions and cached telemetry. - **Do not choose it when:** You require an HTTP-only integration or fields outside its supported representation.

Implementation: OpenF1 - Choose it when: Service-style access to historical/live endpoint data and independent stream retrieval. - **Do not choose it when:** You assume every endpoint is synchronized or identical to FastF1.

Implementation: Both as separate datasets - Choose it when: You want cross-source validation or broader coverage. - **Do not choose it when:** You have not measured semantic and temporal differences.

Implementation: Merge sources into one row - Choose it when: There is a justified key, timing policy and conflict-resolution rule. - **Do not choose it when:** Matching is based only on similar column names.

APEX V1 supports both adapters but does not require both for one run. Each produces the canonical contract; downstream code stays source-agnostic. Synthetic data remains the offline integration-test source.

Transfer the lesson into Project APEX

Run an adapter command when network access is available, save canonical output, validate it, then call the same run-canonical pipeline used for any source.

Repository path to inspect

```
apex_engine/src/apexsim/data/fastf1_adapter.py
apex_engine/src/apexsim/data/openf1_adapter.py
apex_engine/src/apexsim/cli.py
apex_engine/README.md
```

Connection to research

Public datasets are observations, not the world itself. A world model trained on one source can inherit its sampling, filtering and missingness as if those were physical laws. Cross-source tests reveal some of these artifacts.

Check your understanding before continuing

Why should raw source snapshots be immutable?

What should happen when an external field disappears?

Why is network retrieval inside a PyTorch Dataset a bad default?

Solutions and reasoning

1. They allow exact replay, adapter debugging and proof of what the source returned at run time. 2. The adapter should fail clearly or mark the field unavailable according to a versioned policy; silently substituting another field can change meaning. 3. It makes samples nondeterministic, slow, rate-limit dependent and difficult to retry or reproduce.

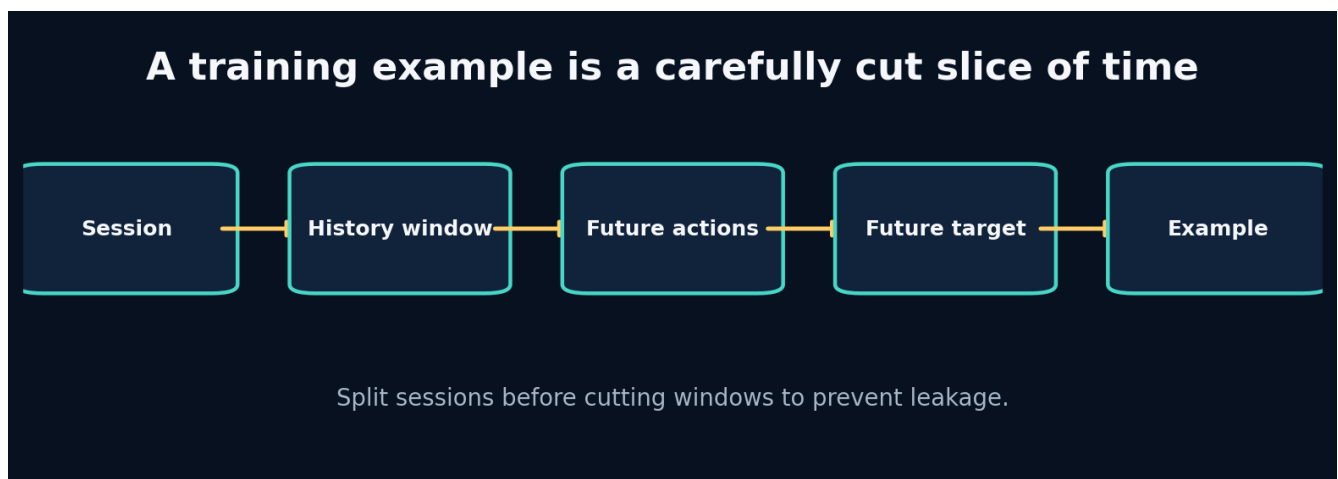
Independent build challenge

Create a recorded fixture for one mocked FastF1-like table and one OpenF1-like JSON response. Write adapters that produce byte-for-byte identical canonical CSV rows.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

7. Windows, Session Splits and Normalization Without Leakage

Instructor objective: Turn canonical sessions into causal training examples while keeping validation and test evidence genuinely independent.



7. Windows, Session Splits and Normalization Without Leakage

The problem that earns this chapter

A telemetry session yields thousands of overlapping windows. If you randomly split those windows, nearly identical frames from the same lap appear in both training and test. The test score becomes a measure of memorization under overlap, not generalization to a new session.

Predict before reading

A 100-frame session uses 20 history frames and 5 future frames with stride 1. How many windows can be created? If windows 1 and 2 are put in different splits, how many frames do they share?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

A sequence example has three distinct pieces: history observations, future interventions/context, and future target state. The model must not receive the future target inside its inputs. This sounds obvious, but dataframe slicing and feature lists can quietly duplicate it.

Split units should match the deployment claim. If the engine will simulate a new race session, split by session or event before making windows. A time-based holdout within one session answers a narrower question. Random windows almost always overstate performance when overlap is high.

Normalization must be fitted only on training data. Validation and test are transformed using stored training statistics. Otherwise the mean and variance leak information about future or held-out distributions, and deployment cannot reproduce the transform.

Vocabulary that now has a job

Concept: History length - Meaning in plain language: How many past frames are visible before a forecast begins. - **Role inside APEX:** Controls memory evidence and input cost.

Concept: Forecast horizon - Meaning in plain language: How many future transitions are predicted. - **Role inside APEX:** Defines simulation duration and error accumulation.

Concept: Split unit - Meaning in plain language: The independent entity assigned wholly to train, validation or test. - **Role inside APEX:** Usually session/event, not overlapping windows.

Concept: Train-only normalizer - Meaning in plain language: Scaling statistics fitted exclusively on training observations. - **Role inside APEX:** Persisted with the model artifact.

Worked example: calculate it by hand

For length $N=100$, history $H=20$, future $F=5$, stride 1, the first window consumes frames 0–24 and the last begins at 75, so the count is

$$N - H - F + 1 = 100 - 20 - 5 + 1 = 76.$$

Window 1 uses frames 0–24; window 2 uses 1–25. They share 24 of 25 frames. Assigning them to different splits creates almost complete leakage.

For normalization, training speeds [40, 50, 60] have mean 50 and standard deviation about 8.165. A test speed 70 becomes $(70-50)/8.165 \approx 2.45$. Recomputing the mean with test data would shrink that value and leak the test distribution.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/08_dataset_windows

What we are about to build

Create history/target windows and inspect shapes and boundaries. Then map the same logic to the production window builder, which also carries future controls and session identifiers.

Runnable implementation

```
import numpy as np
from education.core import sliding_windows

values=np.arange(30,dtype=float).reshape(10,3)
x,y=sliding_windows(values,history=4,horizon=2)
print("x",x.shape,"y",y.shape)
print("first history\n",x[0]); print("first future\n",y[0])
```

Observed output from the packaged solution

Traceback (most recent call last):

File "/mnt/data/Project_APEX_Engineering_Apprenticeship/labs/08_dataset_windows/solution.py", line 2, in <module>

from education.core import sliding_windows

ModuleNotFoundError: No module named 'education'

Read the important lines like English

Code: for start in range(...) - **What the line is doing:** Enumerate valid causal cut points. - **What to**

inspect: The upper bound must leave enough future target frames.

Code: history = data[start:start+H] - **What the line is doing:** Select only evidence available before forecast time. - **What to inspect:** Verify the last history timestamp precedes the first target timestamp.

Code: target = data[start+H:start+H+F] - **What the line is doing:** Select the future trajectory to learn. - **What to inspect:** Do not accidentally include target state in future inputs.

State and tensor trace

session frames: 0 1 2 3 4 5 6 7 8 9

history H=4: [0 1 2 3]

future controls: [4 5 6]

future targets: [4 5 6]

next window: [1 2 3 4] → [5 6 7]

Controls and target states share timestamps but not semantics. Future throttle may be supplied; future speed is what must be predicted.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Create all windows first and randomly split them. Train a nearest-neighbour or linear baseline. Compare its test score with a session-level split.

Diagnose from the earliest failed contract

Inspect source session IDs and frame-index overlap across splits. If any raw session contributes to multiple splits, your claimed generalization unit is false.

Repair and lock the repair with a test

Assign sessions/events deterministically to splits before windowing. Fit normalization on training windows only. Add tests for disjoint identifiers and for target columns absent from model inputs.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Random window split - Choose it when: Only for debugging shape/training code, never for a generalization claim with overlap. - **Do not choose it when:** Any realistic telemetry evaluation.

Implementation: Session split - Choose it when: Deployment targets unseen sessions under similar tracks/drivers. - **Do not choose it when:** You specifically study later portions of the same session.

Implementation: Event/track split - Choose it when: You need evidence of transfer to new circuits or events. - **Do not choose it when:** The dataset is too small to support that claim.

Implementation: Rolling temporal split - Choose it when: Production learns from past sessions and forecasts future dates. - **Do not choose it when:** Timestamp order does not correspond to deployment.

APEX V1 splits by session before creating windows and stores the split assignment. Normalization statistics are versioned artifacts loaded by training, evaluation, simulation and UI.

Transfer the lesson into Project APEX

Trace one WindowDataset sample. Print history, future inputs, targets, session ID, start timestamp and feature names. Then inspect the split tests.

Repository path to inspect

```
apex_engine/src/apexsim/data/windows.py
apex_engine/src/apexsim/pipeline/stages.py
apex_engine/tests/test_windows.py
debugging_cases/04_random_window_split
debugging_cases/06_normalizer_leak
```

Connection to research

World-model papers often report benchmark splits that already define episodes. In custom telemetry projects, defining the episode and independence unit is part of the research contribution, not preprocessing trivia.

Check your understanding before continuing

Why can overlapping windows inflate scores even without exact duplicate rows?

Should validation statistics be used to choose normalization?

What deployment claim does a track holdout support?

Solutions and reasoning

1. They share most temporal context and local dynamics, so the model sees nearly the same trajectory during training. 2. No. Normalization is learned from training only; validation is used for model and hyperparameter selection under that fixed transform. 3. Evidence that the learned dynamics and representation transfer to an unseen circuit distribution, subject to the held-out tracks tested.

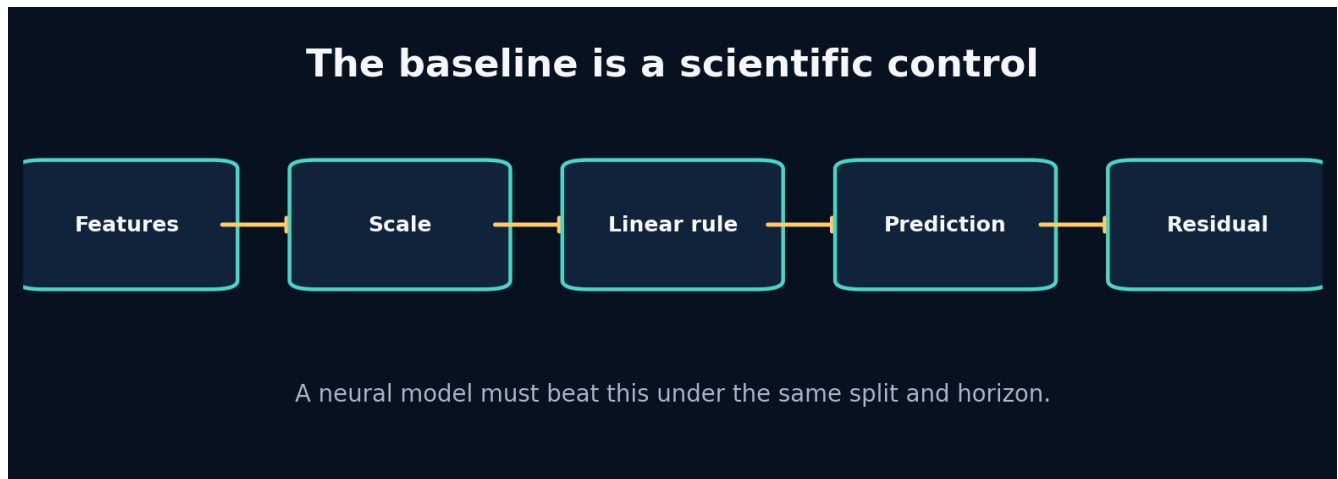
Independent build challenge

Implement a split audit that reports shared session IDs, shared raw frame identifiers, time overlap and distribution differences for every pair of splits.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

8. Statistics, Baselines and Uncertainty as Engineering Controls

Instructor objective: Use simple models and residual analysis to determine what the data contains before escalating to deep sequence architectures.



8. Statistics, Baselines and Uncertainty as Engineering Controls

The problem that earns this chapter

A GRU can produce a low loss for the wrong reason: leaked targets, easy persistence, narrow test conditions or an imbalanced metric. A baseline is the control group that tells you whether complexity has earned its place.

Predict before reading

At 5 Hz, speed changes slowly between adjacent frames. Which will be harder to beat for one-step speed prediction: a constant mean, persistence (next speed = current speed), or a linear action-conditioned model? Explain how the answer may change at a longer horizon.

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

Start with the distribution: means, spreads, missingness, correlations and conditional relationships. Then create baselines that correspond to real hypotheses. Persistence says the world changes slowly. Linear regression says the change is approximately additive in observed features. A physics baseline says known equations explain most of the transition.

Residuals—target minus prediction—show what the baseline failed to explain. Plot residuals against speed, curvature, rain, tyre age and horizon. Structured residuals are a blueprint for the next model. Random-looking residuals near measurement noise may mean a larger model has little room to help.

Uncertainty has several meanings. Aleatoric uncertainty reflects inherently variable outcomes or noisy measurements. Epistemic uncertainty reflects lack of knowledge, often rising outside the training distribution. A point predictor does not automatically communicate either.

Vocabulary that now has a job

Concept: Persistence baseline - Meaning in plain language: Predict that the next state equals the current state. - **Role inside APEX:** Strong short-horizon speed control.

Concept: Residual - Meaning in plain language: Observed target minus model prediction. - **Role inside APEX:** Reveals missing nonlinearities, context or delay.

Concept: Calibration - Meaning in plain language: Predicted uncertainty/probability matches empirical frequency or error. - **Role inside APEX:** Needed before presenting confidence in imagined futures.

Concept: Distribution shift - Meaning in plain language: Deployment evidence differs from training evidence. - **Role inside APEX:** New tracks, weather, drivers or game telemetry.

Worked example: calculate it by hand

Suppose true next speeds are [50.2, 60.1, 69.7] and persistence predicts [50.0, 60.0, 70.0].

Absolute errors are [0.2, 0.1, 0.3]; MAE is $(0.2+0.1+0.3)/3 = 0.2$ m/s.

A neural model with MAE 0.18 improves by 0.02 m/s, or 10% relative. That gain might matter—or it might vanish on a new session, cost far more compute, and introduce physical violations. Compare by horizon, subgroup, latency and stability before declaring victory.

If residuals under braking average +0.8 m/s (targets are higher than predictions), the baseline is overestimating deceleration. That points to brake scaling, delay or grip context.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/05_regression_baseline

What we are about to build

Fit a Ridge transition model to synthetic speed, throttle and brake data, inspect coefficients and calculate test MAE.

Runnable implementation

```
import numpy as np
from sklearn.linear_model import Ridge

rng=np.random.default_rng(4)
speed=rng.uniform(20,80,1000)
throttle=rng.uniform(0,1,1000)
brake=rng.uniform(0,1,1000)
y=speed + 0.7*throttle - 1.1*brake - 0.0008*speed**2 + rng.normal(0,0.05,1000)
X=np.column_stack([speed,throttle,brake])
model=Ridge(alpha=1.0).fit(X[:800],y[:800])
print("coefficients:", model.coef_)
print("test MAE:", np.mean(np.abs(model.predict(X[800:])-y[800:])))
```

Observed output from the packaged solution

```
coefficients: [ 0.91911817  0.68984381 -1.03384695]
test MAE: 0.18453407932904015
```

Read the important lines like English

Code: `X=np.column_stack([speed,throttle,brake])` - **What the line is doing:** Construct one row per transition with named causal inputs. - **What to inspect:** Column order must match stored feature metadata.

Code: `Ridge(alpha=1.0).fit(X[:800],y[:800])` - **What the line is doing:** Estimate a regularized linear transition using training rows. - **What to inspect:** The split here is educational; real telemetry should split by session.

Code: `model.coef_` - **What the line is doing:** Expose the learned direction and relative scale of each feature. - **What to inspect:** Interpret coefficients only after accounting for units and scaling.

State and tensor trace

```
[speed, throttle, brake]
  ↓ linear weighted sum
predicted next speed
  ↓ subtract observed next speed
residual
  ↓ group/plot by condition
model-improvement hypothesis
```

The residual is not merely an error score; it is evidence about missing structure.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Add the target next speed as an input feature and watch MAE collapse. Then randomize brake while preserving speed. Compare coefficient signs and residuals.

Diagnose from the earliest failed contract

A suspiciously perfect baseline should trigger a feature audit and time-index trace before celebration. Check that every input is available at forecast time and that split entities are disjoint.

Repair and lock the repair with a test

Maintain an allow-list of input features, unit-test target exclusion, and report persistence beside every model. Add residual plots by horizon and condition to evaluation artifacts.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Persistence - Choose it when: Very short horizons and slowly changing states. - **Do not choose it when:** Actions or geometry cause rapid transitions.

Implementation: Linear / Ridge - Choose it when: You need interpretability, speed and a strong control. - **Do not choose it when:** Residuals show substantial nonlinear or temporal structure.

Implementation: Gradient-boosted trees - Choose it when: Tabular nonlinearities dominate and fixed windows can summarize history. - **Do not choose it when:** You need smooth autoregressive latent dynamics or differentiable planning.

Implementation: Probabilistic baseline - Choose it when: Forecast intervals or multi-modal outcomes matter. - **Do not choose it when:** You have not first validated deterministic error and calibration.

APEX keeps persistence and a linear transition baseline in the same evaluation harness as learned world models. A complex model is promoted only when it improves the relevant horizons without unacceptable violations or operational cost.

Transfer the lesson into Project APEX

Open the baseline implementation and evaluation report. Add a residual plot against curvature and a table comparing relative improvement over persistence.

Repository path to inspect

```
apex_engine/src/apexsim/models/baselines.py
apex_engine/src/apexsim/evaluation.py
apex_engine/src/apexsim/pipeline/stages.py
debugging_cases/03_target_leakage
```

Connection to research

Strong baselines protect research from architecture-driven conclusions. If a latent world model cannot outperform persistence on the actual decision horizon, its representation may still be useful—but that claim needs a different evaluation.

Check your understanding before continuing

Why is relative improvement over persistence more informative than a standalone MAE?

What does a curved residual-vs-speed plot suggest?

Can uncertainty be low while the prediction is wrong?

Solutions and reasoning

1. It shows how much predictable change the model captures beyond the natural smoothness of telemetry. 2. The transition contains nonlinear speed dependence or the feature scaling/model form is misspecified. 3. Yes. An uncalibrated model can be confidently wrong, especially under distribution shift.

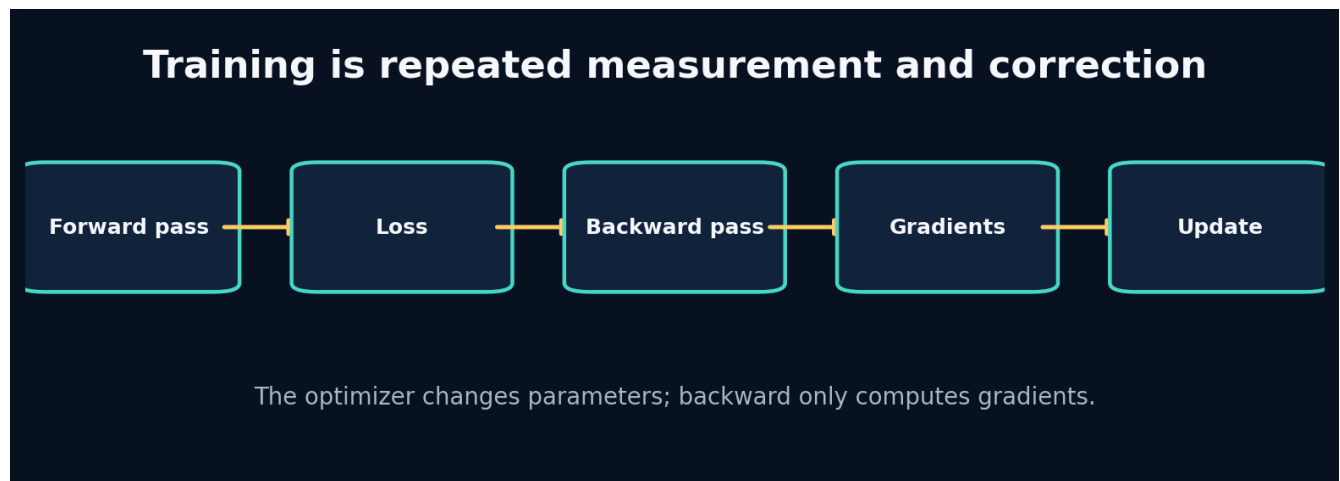
Independent build challenge

Create persistence, linear and polynomial baselines under the same session split. Produce a one-page model card explaining which baseline wins at 1, 5 and 20 steps and why.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

9. Tensors, Autograd and the Training Loop You Can Explain

Instructor objective: Build a PyTorch training loop while understanding every tensor, gradient and parameter update rather than memorizing boilerplate.



9. Tensors, Autograd and the Training Loop You Can Explain

The problem that earns this chapter

Many learners can write `loss.backward()` but cannot say what changed afterward. This becomes dangerous when gradients accumulate, validation tracks graphs, or tensor axes are swapped. We will turn the training loop into an observable state machine.

Predict before reading

Immediately after `loss.backward()` and before `optimizer.step()`, have the model weights changed? Where is the information needed to change them stored? What happens if `zero_grad()` is omitted for two iterations?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

A tensor is an array plus shape, dtype, device and—when requested—a connection to a computation graph. The forward pass applies parameterized operations. The loss reduces prediction error to a scalar. Backpropagation computes derivatives of that scalar with respect to leaf parameters and stores them in `.grad`. The optimizer then uses those gradients to update parameters.

Gradients accumulate by default. That enables deliberate gradient accumulation but causes a bug when forgotten. Evaluation must switch behaviour-sensitive layers with `model.eval()` and disable graph construction with `torch.no_grad()`.

The most effective way to learn this is to print parameter values, gradients and loss before and after each line on a tiny problem whose correct mapping is known.

Vocabulary that now has a job

Concept: Computation graph - Meaning in plain language: The recorded chain of operations connecting inputs, parameters and loss. - **Role inside APEX:** Enables gradient-based training of all world models.

Concept: Gradient - Meaning in plain language: Local sensitivity of loss to a parameter. - **Role inside APEX:** Stored in each trainable parameter after backward.

Concept: Optimizer state - Meaning in plain language: Extra memory used to turn gradients into updates. - **Role inside APEX:** Adam moments are checkpointed with the model during resumable training.

Concept: Train/eval mode - Meaning in plain language: Switch for layers such as dropout and batch normalization. - **Role inside APEX:** Ensures deterministic, correctly normalized validation and rollout.

Worked example: calculate it by hand

For a one-parameter model $\hat{y} = w \times x$ with $x=2$, $y=6$, $w=1$, and squared loss $((y-\hat{y})^2$:

Prediction: $1 \times 2 = 2$.

Error: $2 - 6 = -4$.

Loss: 16.

Gradient: $dL/dw = 2(wx - y)x = 2(-4)(2) = -16$.

With learning rate 0.1, SGD update: $w \leftarrow 1 - 0.1(-16) = 2.6$.

New prediction: 5.2; new loss: 0.64.

Backward computes -16 ; the optimizer performs the update to 2.6.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/10_training_loop

What we are about to build

Train a linear PyTorch module to sum three input features. Print training loss, gradient norm and validation loss each epoch.

Runnable implementation

```
import torch
from torch import nn
```

```
torch.manual_seed(1); x=torch.randn(128,3); y=x.sum(1,keepdim=True)
model=nn.Linear(3,1); opt=torch.optim.Adam(model.parameters(),lr=0.05)
for epoch in range(8):
    model.train(); opt.zero_grad(); pred=model(x); loss=((pred-y)**2).mean(); loss.backward()
    grad=float(model.weight.grad.norm()); opt.step()
    model.eval();
    with torch.no_grad(): val=((model(x)-y)**2).mean()
    print(epoch,float(loss),grad,float(val))
```

Observed output from the packaged solution

```
0 3.2264039516448975 3.7724902629852295 2.9054489135742188
1 2.9054489135742188 3.5811667442321777 2.6067750453948975
2 2.6067750453948975 3.390789270401001 2.3303141593933105
3 2.3303141593933105 3.2017178535461426 2.0752670764923096
4 2.0752670764923096 3.0143868923187256 1.8400570154190063
5 1.8400570154190063 2.829359292984009 1.6230223178863525
6 1.6230223178863525 2.6473701000213623 1.4232583045959473
7 1.4232583045959473 2.469295024871826 1.2406574487686157
```

/mnt/data/Project_APEX_Engineering_Apprenticeship/labs/10_training_loop/solution.py:11:

UserWarning: Converting a tensor with requires_grad=True to a scalar may lead to unexpected behavior.

Consider using tensor.detach() first. (Triggered internally at

/pytorch/torch/csrc/autograd/generated/python_variable_methods.cpp:836.)

```
print(epoch,float(loss),grad,float(val))
```

Read the important lines like English

Code: opt.zero_grad() - **What the line is doing:** Clear gradients left by previous backward passes. -

What to inspect: It does not reset model weights.

Code: pred=model(x) - **What the line is doing:** Run the forward computation using current parameters. - **What to inspect:** Inspect prediction shape against target shape.

Code: loss.backward() - **What the line is doing:** Populate .grad for parameters connected to the loss. -

What to inspect: Weights have not yet changed.

Code: opt.step() - **What the line is doing:** Use gradients and optimizer state to update parameters. -

What to inspect: Compare parameter values before and after.

Code: with torch.no_grad() - **What the line is doing:** Compute validation outputs without constructing a backward graph. - **What to inspect:** Also call model.eval() for mode-dependent layers.

State and tensor trace

parameters θ_0

↓ forward

predictions $\hat{y}$

↓ loss

scalar L

↓ backward

θ .grad populated; θ still equals θ_0

↓ optimizer.step

parameters θ_1

Print `id(parameter)`, `parameter.detach()`, and `parameter.grad` around these steps once. That trace removes the mystery permanently.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Remove `zero_grad`, increase learning rate by 100×, and accidentally shape targets as `[B]` while predictions are `[B,1]`. Observe accumulation, divergence and broadcasting.

Diagnose from the earliest failed contract

Check shapes first, then finite values, loss scale, gradient norms, parameter-update magnitude and mode. Debug the smallest batch before scaling to full telemetry.

Repair and lock the repair with a test

Add assertions for exact prediction/target shape, finite loss and finite gradients. Add a one-batch overfit test and a deterministic seed. Make gradient accumulation an explicit configured feature if used.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: SGD - Choose it when: You want transparent updates and can tune schedules carefully. - **Do not choose it when:** Sparse/noisy gradients need faster adaptive progress.

Implementation: Adam/AdamW - Choose it when: A strong default for deep sequence models and fast iteration. - **Do not choose it when:** You assume it removes the need for learning-rate and regularization experiments.

Implementation: Gradient clipping - Choose it when: Recurrent or long-horizon training shows occasional exploding norms. - **Do not choose it when:** It hides systematically bad scaling or unstable architecture.

Implementation: Mixed precision - Choose it when: GPU throughput/memory matters and numerical checks pass. - **Do not choose it when:** You have not established a stable full-precision reference.

APEX training records seed, configuration, gradient-related settings, checkpoint metric and normalizer. Validation is performed under eval/no-grad and the best checkpoint is restored before test evaluation.

Transfer the lesson into Project APEX

Trace one batch through the GRU training function. Print every shape and compare parameter checksum before backward, after backward and after step.

Repository path to inspect

`apex_engine/src/apexsim/training.py`

`apex_engine/src/apexsim/models/gru_world_model.py`

debugging_cases/08_missing_eval_mode
 debugging_cases/09_gradient_accumulation

Connection to research

Research papers summarize optimization in a paragraph, but reproducing them depends on these details: gradient scale, target construction, optimizer state, evaluation mode, clipping, schedules and checkpoint selection.

Check your understanding before continuing

What exactly does `backward()` mutate?

Why can broadcasting produce a low-looking loss with the wrong semantics?

What does successful one-batch overfitting prove and not prove?

Solutions and reasoning

1. It accumulates gradients into `.grad` fields of leaf tensors requiring gradients; it does not update parameter values. 2. PyTorch expands compatible dimensions silently, comparing unintended pairs while still returning a scalar. 3. It proves the data path, model and optimizer can memorize a tiny sample; it does not prove generalization, correct splits or realistic rollout behaviour.

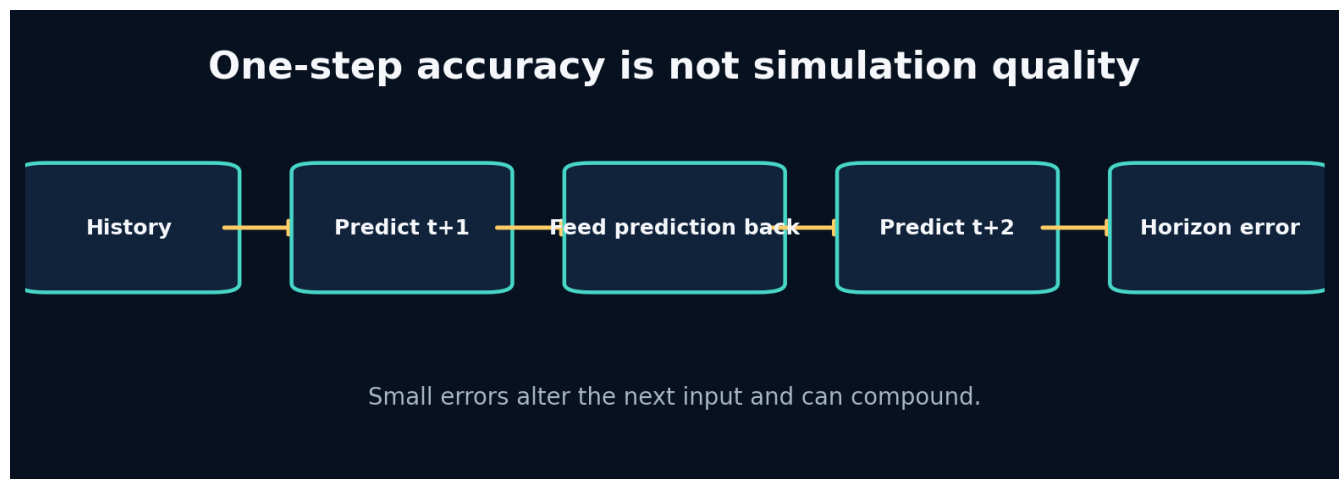
Independent build challenge

Instrument a training step to write a JSON trace containing input shapes, loss, gradient norms and parameter-update norms. Use it to diagnose three intentionally broken configurations.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

10. Transition Models and Why Rollouts Fail

Instructor objective: Build a one-step neural transition, convert it into an autoregressive simulator, and measure error compounding honestly.



10. Transition Models and Why Rollouts Fail

The problem that earns this chapter

A model can be excellent at predicting the next frame when given the true current frame, yet collapse when asked to simulate ten frames. During rollout it consumes its own imperfect predictions, a distribution it did not see during teacher-forced training.

Predict before reading

A model has a constant +0.1 m/s bias per predicted step and no corrective feedback. What speed bias do you expect after 20 autoregressive steps? What if the transition also multiplies the current error by 1.05 each step?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

One-step supervised training samples inputs from real data. Autoregressive inference samples later inputs from the model. This train–inference distribution shift is exposure bias. Even unbiased local noise can alter future controls' effects because the state enters nonlinear dynamics.

A rollout function must specify which future variables are known and which are predicted. In APEX, future controls/context can be provided by a scenario, while future state is generated. After each step, predicted state replaces the state portion of the next input; controls remain externally supplied.

Evaluation should report error by horizon rather than averaging all future steps. The shape of the curve distinguishes immediate bias, unstable compounding and long-memory failure.

Vocabulary that now has a job

Concept: Teacher forcing - **Meaning in plain language:** Train each transition using the true previous state. - **Role inside APEX:** Stable supervised training for GRU/RSSM components.

Concept: Autoregressive rollout - **Meaning in plain language:** Feed predicted state back to generate later states. - **Role inside APEX:** The actual simulation mode.

Concept: Exposure bias - **Meaning in plain language:** The model trains on true states but runs on its own imperfect states. - **Role inside APEX:** A central source of horizon degradation.

Concept: Horizon curve - **Meaning in plain language:** Error reported separately at each future step. - **Role inside APEX:** Primary evidence for useful simulation duration.

Worked example: calculate it by hand

With additive bias $e_{t+1}=e_t+0.1$, starting at zero, error after 20 steps is 2.0 m/s.

With amplification $e_{t+1}=1.05e_t+0.1$,

$$e_{20}=0.1 \frac{1.05^{20}-1}{1.05-1} \approx 3.31 \text{ m/s}.$$

The one-step bias is still only 0.1, but the system dynamics amplify it. This is why one-step MAE cannot certify a simulator.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/07_pytorch_module

What we are about to build

Use a small PyTorch transition module as the building block, then write a loop that feeds predicted state back while taking future controls from an external sequence.

Runnable implementation

```
import torch
from torch import nn

class Transition(nn.Module):
    def __init__(self):
        super().__init__()
        self.net=nn.Sequential(nn.Linear(5,16),nn.ReLU(),nn.Linear(16,2))
    def forward(self,x): return self.net(x)
```

```
model=Transition(); x=torch.randn(4,5); y=model(x)
print(model); print("input",x.shape,"output",y.shape)
```

Observed output from the packaged solution

```
Transition(
  (net): Sequential(
    (0): Linear(in_features=5, out_features=16, bias=True)
    (1): ReLU()
    (2): Linear(in_features=16, out_features=2, bias=True)
  )
)
input torch.Size([4, 5]) output torch.Size([4, 2])
```

Read the important lines like English

Code: `prediction = model(current_input)` - **What the line is doing:** Predict the next state from current state/action/context. - **What to inspect:** Confirm output contains only target state variables.

Code: `current_state = prediction` - **What the line is doing:** Move the imagined world forward using its own result. - **What to inspect:** Detach only when appropriate; training through rollout may need gradients.

Code: `next_input = concat(current_state, future_control[t])` - **What the line is doing:** Combine generated state with externally specified intervention. - **What to inspect:** Never overwrite the future control with a predicted target column.

State and tensor trace

real history → encoded state

step 1: predicted state₁ + supplied action₁ → input₂

step 2: predicted state₂ + supplied action₂ → input₃

step 3: predicted state₃ + supplied action₃ → input₄

At each step, log both normalized and physical-unit state. Violations may be hidden in normalized space.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

During rollout, accidentally keep feeding the last real state instead of each prediction. The multi-step score may look surprisingly strong because you are not actually simulating.

Diagnose from the earliest failed contract

Trace the source of every input tensor at every horizon. Mark each element as observed, supplied intervention, static context or model-generated. A rollout audit catches “teacher forcing at test time.”

Repair and lock the repair with a test

Add a test with a deterministic transition where the exact multi-step trajectory is known. Assert that changing a predicted state changes later predictions. Report teacher-forced and free-running metrics separately.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: One-step training - Choose it when: A stable starting point with abundant transition examples. - **Do not choose it when:** You treat its metric as rollout certification.

Implementation: Multi-step loss - Choose it when: Long-horizon accuracy matters and compute allows unrolled training. - **Do not choose it when:** Early training is unstable or the horizon curriculum is not controlled.

Implementation: Scheduled sampling - Choose it when: You want gradual exposure to model-generated states. - **Do not choose it when:** You assume it guarantees consistent probabilistic learning.

Implementation: Direct multi-horizon model - Choose it when: Fixed-horizon forecasts matter more than a reusable transition. - **Do not choose it when:** You need arbitrary-length interactive simulation.

APEX trains a stable sequence world model and evaluates it under free-running rollouts. Any reported simulation horizon is tied to its error and violation curves, not a marketing duration.

Transfer the lesson into Project APEX

Inspect `simulation.py` and identify the exact line where predicted state is inserted into the next model input. Add a debug mode that records provenance for every future feature.

Repository path to inspect

```
projects/04_mlp_transition/main.py
apex_engine/src/apexsim/simulation.py
apex_engine/src/apexsim/evaluation.py
apex_engine/src/apexsim/models/gru_world_model.py
```

Connection to research

World-model planning magnifies rollout weaknesses because a planner searches for trajectories that exploit them. Reliable imagination requires evaluating model-generated state distributions, not only posterior-conditioned reconstruction.

Check your understanding before continuing

Why can zero-mean one-step noise still create biased long rollouts?

What future variables may legitimately be supplied to a simulator?

What does a flat horizon-error curve suggest?

Solutions and reasoning

1. Nonlinear dynamics and constraints can transform symmetric local noise into asymmetric state evolution. 2. Known scenario interventions and exogenous forecasts, provided their availability is explicit; future target state cannot be supplied. 3. The model may be stable over that range, or errors may be dominated by a constant initial bias rather than compounding; inspect conditions and baselines.

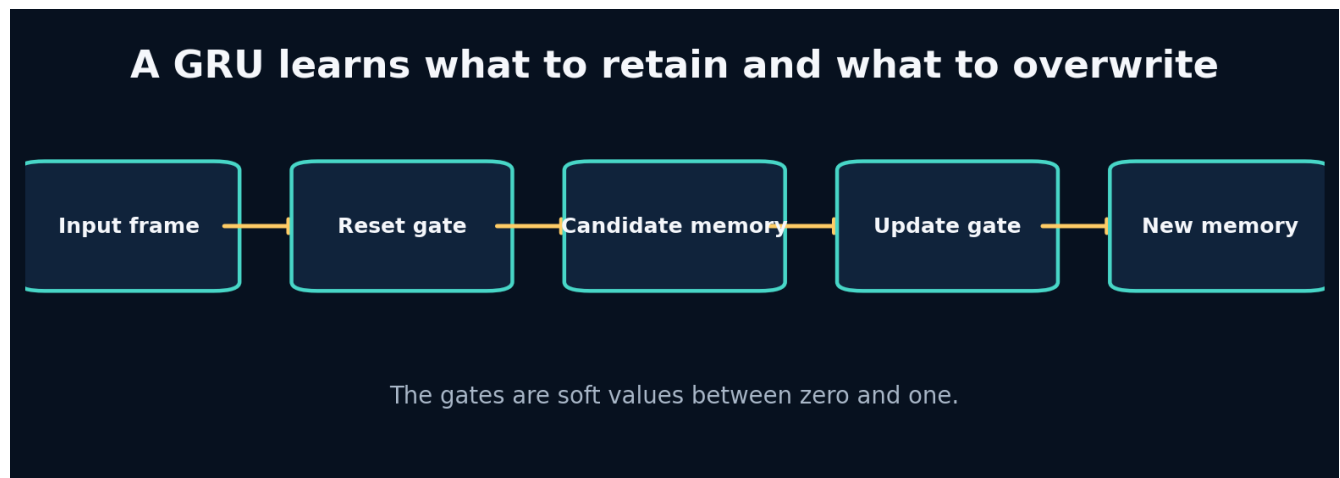
Independent build challenge

Train the same transition with one-step and five-step losses. Compare horizon curves, training stability and compute. Record which objective you choose for APEX V1 and why.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

11. Recurrent Memory and the GRU From the Inside

Instructor objective: Earn recurrent memory by exposing the limits of fixed-frame transitions, then inspect how GRU gates update hidden state.



11. Recurrent Memory and the GRU From the Inside

The problem that earns this chapter

Current speed, throttle and brake may not reveal whether the car has been accelerating for two seconds, exiting a corner, heating tyres or recovering from a braking event. A fixed-frame model sees only the present. We need a mechanism that carries a learned summary of the past.

Predict before reading

Two sequences end with the same final frame [speed=50, throttle=0.5, brake=0]. Sequence A accelerated steadily; sequence B braked hard and recovered. Should a recurrent model be able to produce different next-state predictions? What must differ internally?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

A recurrent neural network updates hidden memory as each frame arrives. The final hidden state is not a literal copy of history; it is a learned summary optimized for the training objective. A vanilla recurrent cell can struggle when useful information must survive many updates because gradients repeatedly pass through the same transition.

A GRU adds gates. The update gate decides how much old memory to keep versus replace. The reset gate controls how much previous memory influences the candidate update. These are soft, feature-wise decisions, not binary switches. The same hidden dimension can retain long-term tyre information while rapidly updating braking state.

Using nn.GRU does not guarantee meaningful memory. You still need enough history, correct ordering, no hidden-state leakage across independent sequences, and evaluation that tests whether memory improves the target horizon.

Vocabulary that now has a job

Concept: Hidden state - Meaning in plain language: A learned vector carried across sequence steps. -

Role inside APEX: Compressed recent driving and unobserved dynamical context.

Concept: Update gate - Meaning in plain language: Controls interpolation between old memory and candidate memory. - **Role inside APEX:** Lets some information persist while other information changes.

Concept: Reset gate - Meaning in plain language: Controls how previous memory contributes to the candidate update. - **Role inside APEX:** Allows rapid forgetting when current evidence changes regime.

Concept: Sequence boundary - Meaning in plain language: The point where hidden memory must be reset or intentionally transferred. - **Role inside APEX:** Driver/session windows cannot share hidden state accidentally.

Worked example: calculate it by hand

A simplified update is

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$

If old memory is $h=0.8$, candidate memory is -0.2 :

With update gate $z=0.1$: $h_t=0.9(0.8)+0.1(-0.2)=0.70$. Most old memory survives.

With $z=0.9$: $h_t=0.1(0.8)+0.9(-0.2)=-0.10$. The new evidence largely overwrites memory.

This interpolation makes a gate interpretable at the mechanism level, although individual learned dimensions may not map cleanly to human concepts.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/12_gru_gates

What we are about to build

Feed three simple one-hot-like inputs through a GRUCell and print hidden state after each step. The goal is to see memory as a changing vector rather than a black box.

Runnable implementation

import torch

```
torch.manual_seed(2); cell=torch.nn.GRUCell(2,4); h=torch.zeros(1,4)
for x in [torch.tensor([[1,0]]),torch.tensor([[0,1]]),torch.tensor([[0,1]])]:
    h=cell(x,h); print(h.detach().numpy().round(3))
```

Observed output from the packaged solution

```
[[ 0.354 -0.151 -0.384  0.149]]
[[ 0.278  0.282 -0.371  0.408]]
[[ 0.322  0.39  -0.39  0.542]]
```

Read the important lines like English

Code: cell=torch.nn.GRUCell(2,4) - **What the line is doing:** Create a transition from a 2-feature input and 4-value memory to new memory. - **What to inspect:** The hidden size is model capacity, not a known physical variable count.

Code: h=torch.zeros(1,4) - **What the line is doing:** Initialize memory for one sequence. - **What to inspect:** Initial-state policy affects early predictions.

Code: h=cell(x,h) - **What the line is doing:** Combine current observation and previous memory into next memory. - **What to inspect:** Sequence order changes the result.

State and tensor trace

$$\begin{array}{lcl} x_1 + h_0=0 & \rightarrow & h_1 \\ x_2 + h_1 & \rightarrow & h_2 \\ x_3 + h_2 & \rightarrow & h_3 \\ & \downarrow & \\ & \text{decoder predicts future} & \end{array}$$

For batched sequences, PyTorch commonly uses [batch, time, features] when batch_first=True. The returned hidden state is typically [layers, batch, hidden]. Name these axes every time.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Reuse the hidden state from one session as the initial state for an unrelated session. Also shuffle time within each sequence while preserving rows.

Diagnose from the earliest failed contract

If validation varies with dataloader order or batch grouping, inspect hidden-state lifetime. Hidden memory should reset at independent sequence boundaries unless stateful streaming is an explicit design.

Repair and lock the repair with a test

Add a test where two sequences processed separately match the same sequences processed in a batch with zero initial states. Add session-boundary reset logic and make stateful mode explicit.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Vanilla RNN - Choose it when: Tiny educational tasks and very short dependencies. - **Do not choose it when:** Long/noisy telemetry histories.

Implementation: GRU - Choose it when: Strong compact recurrent baseline with fewer parameters than LSTM. - **Do not choose it when:** You have evidence that longer structured memory or parallel training is required.

Implementation: LSTM - Choose it when: You value an explicit cell state and gates and have sufficient compute/data. - **Do not choose it when:** Extra complexity provides no measured gain.

Implementation: Temporal convolution - Choose it when: Dependencies fit a fixed receptive field and parallel computation matters. - **Do not choose it when:** You need adaptive unbounded recurrence or stateful streaming.

The GRU is APEX V1's primary deterministic world model because it is understandable, stable, compact and strong enough to establish the pipeline. SSM and RSSM models are challengers, not automatic replacements.

Transfer the lesson into Project APEX

Trace the GRU world model from history encoder to autoregressive decoder. Print hidden shape, decoder input shape and output state at each horizon for a batch of one.

Repository path to inspect

```
labs/11_rnn_from_scratch/solution.py
labs/12_gru_gates/solution.py
projects/06_gru_forecaster/main.py
apex_engine/src/apexsim/models/gru_world_model.py
debugging_cases/07_hidden_state_reuse
```

Connection to research

Recurrent state is a deterministic belief summary. Dreamer's RSSM retains a recurrent deterministic state while adding a stochastic latent variable to represent uncertainty and ambiguity.

Check your understanding before continuing

Why can two sequences with the same final observed frame produce different hidden states?

Does a larger hidden size always improve memory?

When should hidden state persist across API requests?

Solutions and reasoning

1. The recurrence applies ordered updates, so earlier frames alter the memory entering later frames. 2. No. It increases capacity but may overfit, slow training or store irrelevant detail; history, objective and optimization also limit memory. 3. Only in an explicitly stateful stream with stable entity/session identity, ordering guarantees and reset rules. Stateless scenario requests should rebuild from supplied history.

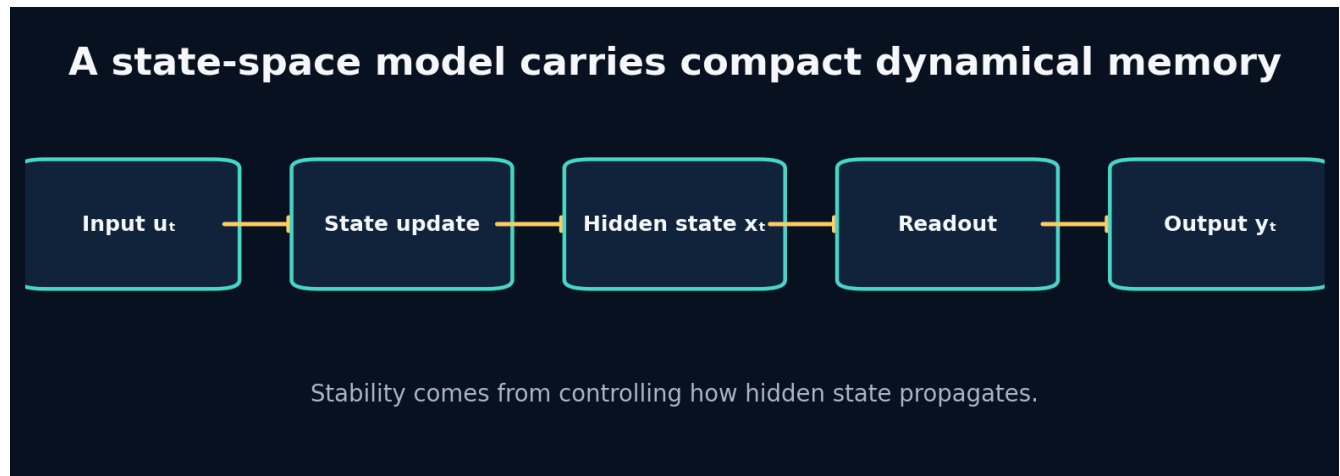
Independent build challenge

Implement a GRU forecaster and a frame-only MLP with matched parameter counts. Create a synthetic target dependent on an event ten steps ago and compare performance as history length changes.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

12. Classical State-Space Models: Dynamics as Memory

Instructor objective: Understand state-space recurrence mathematically, inspect stability, and connect classical dynamical systems to modern sequence models.



12. Classical State-Space Models: Dynamics as Memory

The problem that earns this chapter

GRUs learn memory through generic gates. State-space models begin with an explicit dynamical view: a hidden state evolves under a transition and receives input. This structure can represent long memory efficiently, but an unstable transition can explode over repeated steps.

Predict before reading

For the scalar recurrence $x_t = 1.05 x_{t-1} + u_t$, what happens after inputs become zero? What changes if the transition coefficient is 0.95?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

A discrete linear state-space model is

$$x_t = A x_{t-1} + B u_t, y_t = C x_t + D u_t.$$

The hidden state x is memory; A controls how it persists and mixes; B writes input; C reads output. Repeated powers of A determine whether past information decays, persists, oscillates or explodes.

Many modern SSMs begin from continuous-time dynamics and discretize them for sampled sequences. This connects model behaviour to the time step. A change from 5 Hz to 20 Hz should not be treated as an arbitrary tensor resize when the transition is supposed to represent time.

Linear recurrence alone cannot model all nonlinear racing dynamics, but it exposes memory timescales and stability more directly than a generic black box.

Vocabulary that now has a job

Concept: State matrix A - Meaning in plain language: Controls hidden-state propagation between steps. - **Role inside APEX:** Determines memory decay, coupling and stability.

Concept: Input matrix B - Meaning in plain language: Controls how current telemetry writes into memory. - **Role inside APEX:** Maps actions/context into dynamical state.

Concept: Readout C - Meaning in plain language: Maps hidden memory to predicted output. - **Role inside APEX:** Produces next-state features from SSM memory.

Concept: Spectral radius - Meaning in plain language: Largest absolute eigenvalue of A ; a key discrete stability indicator. - **Role inside APEX:** Values above one can amplify state without bound.

Worked example: calculate it by hand

For $x_t = a x_{t-1}$ after input stops:

If $a=1.05$, then after 20 steps x is multiplied by $1.05^{20} \approx 2.65$.

If $a=0.95$, it is multiplied by $0.95^{20} \approx 0.358$.

If $a=1$, memory persists exactly.

If $a=-0.95$, memory alternates sign while decaying.

For a matrix, eigenvalues play the role of these scalar factors along different modes. Stability does not guarantee usefulness: a transition that decays too quickly forgets everything.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/13_linear_ssm

What we are about to build

Run a two-dimensional linear SSM under an input pulse and observe one state component react immediately while another integrates it more slowly.

Runnable implementation

```
import numpy as np
A=np.array([[0.95,0.0],[0.1,0.85]]); B=np.array([[0.2],[0.05]]); x=np.zeros(2)
```

Hands-on world-model engineering • Predict → Build → Run → Observe → Break → Explain → Choose

```
for u in [1,1,0,0,0]:
    x=A@x+B[:,0]*u; print(x.round(3))
```

Observed output from the packaged solution

```
[0.2 0.05]
[0.39 0.112]
[0.37 0.135]
[0.352 0.151]
[0.334 0.164]
```

Read the important lines like English

Code: `A=np.array([[0.95,0.0],[0.1,0.85]])` - **What the line is doing:** Define decay and coupling of two hidden modes. - **What to inspect:** Diagonal values set persistence; off-diagonal 0.1 transfers state.

Code: `B=np.array([[0.2],[0.05]])` - **What the line is doing:** Define how the scalar input writes into both modes. - **What to inspect:** Different modes receive different input strength.

****Code:** `x=A@x+B[:,0]*u` - **What the line is doing:** **Apply one state-space transition.** - **What to inspect:**** Repeated multiplication reveals stability and memory.

State and tensor trace

$u_0=1 \rightarrow$ immediate mode rises strongly; slow mode rises slightly
 $u_1=1 \rightarrow$ both accumulate
 $u_2=0 \rightarrow$ immediate mode decays; slow mode still receives coupled memory
 $u_3=0 \rightarrow$ both decay on different timescales

Plot each hidden dimension. A hidden state is useful because its dynamics differ, not because it has many numbers.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Change a diagonal value in A to 1.2 and roll for 100 zero-input steps. Then set both diagonal values to 0.1.

Diagnose from the earliest failed contract

Measure hidden-state norm and eigenvalues. Explosion points to unstable propagation; rapid collapse points to memory that decays faster than the required dependency.

Repair and lock the repair with a test

Parameterize stable decay, constrain eigenvalues or add normalization/clipping as appropriate. Add tests that hidden state stays finite under bounded inputs for the intended horizon.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Fixed linear SSM - **Choose it when:** System identification, interpretability and approximately linear dynamics. - **Do not choose it when:** Strong context-dependent nonlinear transitions dominate.

Implementation: Nonlinear readout - Choose it when: Memory dynamics are simple but state-to-output mapping is nonlinear. - **Do not choose it when:** Input must alter the memory law itself.

Implementation: Input-dependent SSM - Choose it when: Different telemetry regimes require selective writing/forgetting. - **Do not choose it when:** A fixed transition already works and simplicity matters.

Implementation: GRU - Choose it when: You need a proven generic nonlinear recurrent baseline. - **Do not choose it when:** Long-sequence efficiency or structured timescales are the measured bottleneck.

APEX includes a small SSM-style challenger with bounded decay and input-dependent updates. It is taught as a selective recurrent model, not represented as a complete optimized Mamba implementation.

Transfer the lesson into Project APEX

Inspect the SSM cell parameters and plot hidden norms across a full session. Compare horizon curves and latency with the GRU under the same data and budget.

Repository path to inspect

```
labs/13_linear_ssm/solution.py
projects/07_selective_ssm/main.py
apex_engine/src/apexsim/models/ssm_world_model.py
debugging_cases/10_exploding_ssm
```

Connection to research

S4 and related structured SSMs use mathematical parameterizations and efficient convolution/recurrent forms to handle long sequences. The classical equation here is the conceptual root that makes those designs readable.

Check your understanding before continuing

Why can a stable SSM still be a bad memory model?

What does an off-diagonal element of A do?

How does sample rate affect a transition intended to represent physical time?

Solutions and reasoning

1. It may decay too quickly, ignore relevant inputs or lack nonlinear/selective behaviour. 2. It transfers information from one hidden component into another during propagation. 3. A smaller time step represents less elapsed time per transition; continuous-time parameterization/discretization can adjust accordingly, while a fixed discrete A changes physical meaning.

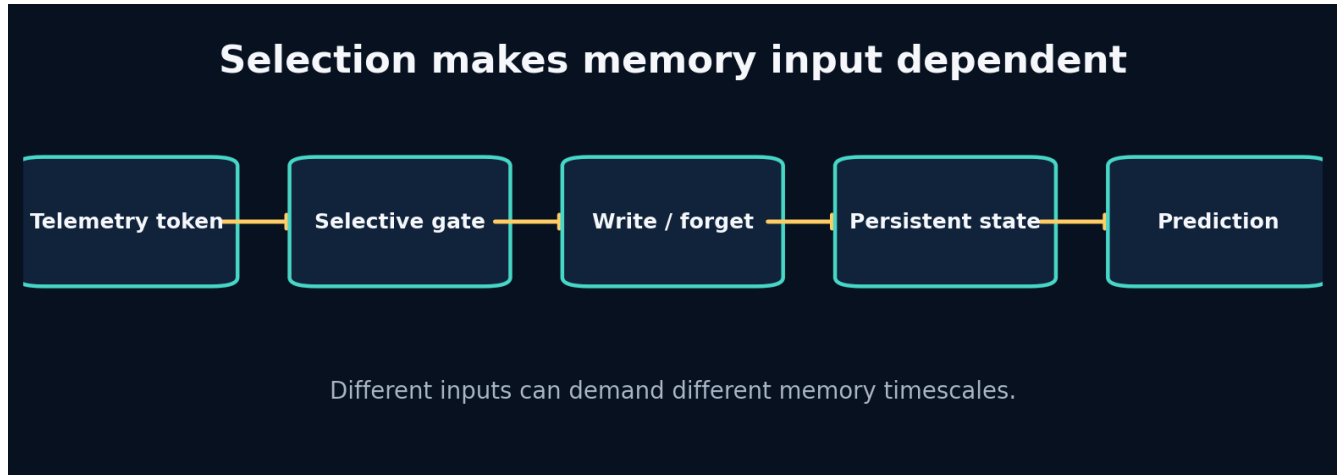
Independent build challenge

Construct a three-mode SSM with fast, medium and slow decay. Feed braking pulses of different duration and show which hidden mode retains each timescale.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

13. Selective State-Space Models and What Mamba Changes

Instructor objective: Build an input-dependent memory cell, understand selection, and separate the core Mamba idea from implementation hype.



13. Selective State-Space Models and What Mamba Changes

The problem that earns this chapter

A fixed SSM remembers every input according to the same transition. Racing telemetry is regime dependent: a yellow flag, braking spike or rain change may deserve a strong memory write, while repetitive straight-line frames may be compressed or forgotten.

Predict before reading

Imagine a memory of “recent heavy braking.” Should the same decay apply during ten ordinary straight-line frames and during a sudden lock-up indicator? How could the current input alter write and forget behaviour?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

Selection means model parameters or gates depend on the current input. Instead of one fixed A and B, the sequence determines how strongly memory persists and what content is written. This gives content-aware recurrence while preserving a state-space perspective.

Mamba combines selective state-space dynamics with hardware-aware computation so long sequences can be processed efficiently. The important conceptual move for this curriculum is not “replace everything with Mamba.” It is: **make memory propagation conditional on the token when fixed dynamics are insufficient.**

A small selective cell can be written as $h_t = a(x_t) \odot h_{t-1} + (1 - a(x_t)) \odot b(x_t)$. It resembles a gated recurrent update. The difference between educational cell and research architecture must remain explicit.

Vocabulary that now has a job

Concept: Selection - Meaning in plain language: Current input changes how memory is written, retained or read. - **Role inside APEX:** Braking, weather or geometry can trigger different memory behaviour.

Concept: Input-dependent decay - Meaning in plain language: The retention factor is computed from the current token. - **Role inside APEX:** Adaptive timescales across race regimes.

Concept: Scan - Meaning in plain language: Efficiently apply a recurrence across sequence positions. - **Role inside APEX:** Important for long histories and GPU throughput.

Concept: Mamba block - Meaning in plain language: A complete architecture combining projections, selective SSM computation and implementation details. - **Role inside APEX:** A future challenger; not identical to the toy selective cell.

Worked example: calculate it by hand

Let old memory be $h=0.9$.

Ordinary input produces retention $a=0.95$ and candidate $b=0.2$: $h'=0.95(0.9)+0.05(0.2)=0.865$.

Critical braking input produces retention $a=0.2$ and candidate $b=-0.8$: $h'=0.2(0.9)+0.8(-0.8)=-0.46$.

The same memory dimension changes slowly during routine input and rapidly under a meaningful event. Selection is useful only if training learns gates that correspond to predictive needs rather than arbitrary noise.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/14_selective_ssm

What we are about to build

Implement a small PyTorch selective cell where decay and candidate write are functions of each input token. Feed basis vectors and inspect hidden updates.

Runnable implementation

```
import torch
from torch import nn

class SelectiveCell(nn.Module):
    def __init__(self,d):
        super().__init__(); self.decay=nn.Linear(d,d); self.write=nn.Linear(d,d)
    def forward(self,x,h):
        a=torch.sigmoid(self.decay(x))*0.99
        b=torch.tanh(self.write(x))
        return a*h+(1-a)*b
cell=SelectiveCell(3); h=torch.zeros(1,3)
for x in torch.eye(3): h=cell(x[None],h); print(h.detach().numpy().round(3))
```

Observed output from the packaged solution

```
[[[-0.114 -0.033 -0.16 ]]  
[[ 0.203  0.146 -0.244]]  
[[ 0.335 -0.015 -0.351]]
```

Read the important lines like English

****Code:** `a=torch.sigmoid(self.decay(x))*0.99` - What the line is doing: **Compute bounded input-dependent retention.** - What to inspect: ****** Multiplying by 0.99 prevents exact unit retention in this toy cell.

Code: `b=torch.tanh(self.write(x))` - **What the line is doing:** Compute candidate content to write. - **What to inspect:** Tanh bounds the candidate but can saturate.

Code: `return ah+(1-a)b` - **What the line is doing:** Interpolate between persistent memory and new content per dimension. - **What to inspect:** Inspect gate distributions across regimes.

State and tensor trace

```
input x_t  
├─> decay network ─> retention a_t ┐  
└─> write network ─> candidate b_t ─┬─> h_t  
previous h_{t-1} ─────────────────┘
```

Log a_t by feature regime. A gate that is always 0.5 is technically selective but may not be learning meaningful specialization.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Remove the sigmoid/bound and allow retention greater than one. Then initialize decay so gates saturate near zero or one.

Diagnose from the earliest failed contract

Track hidden norm, gate histograms, gradient norms and horizon errors. Explosion, frozen memory and always-overwritten memory have different signatures.

Repair and lock the repair with a test

Use stable parameterization, sensible initialization and regular diagnostic plots. Add tests for finite bounded-state behaviour under bounded inputs, but do not force gates to a preferred distribution without evidence.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Toy selective cell - **Choose it when:** Learning the mechanism and testing whether input-dependent memory helps. - **Do not choose it when:** Claiming Mamba-equivalent speed or quality.

Implementation: Official Mamba implementation - **Choose it when:** Long sequences and benchmarks justify the dependency and hardware path. - **Do not choose it when:** Your data are small, horizons short, or environment support is fragile.

Implementation: GRU - Choose it when: A stable gated recurrent baseline is sufficient. - **Do not choose it when:** Measured long-context/throughput limitations motivate another architecture.

Implementation: Transformer - Choose it when: Global content interactions and parallel token processing dominate. - **Do not choose it when:** Quadratic attention cost is unjustified and recurrent deployment state is useful.

APEX V1 retains a GRU primary and an SSM-style challenger. Fine-tuning or replacing with a full Mamba model should begin only after sequence length, data scale and latency measurements establish the need.

Transfer the lesson into Project APEX

Compare GRU and SSM models with identical windows, splits, normalization, hidden size budget and evaluation. Add gate plots to the SSM run artifacts.

Repository path to inspect

```
labs/14_selective_ssm/solution.py
projects/07_selective_ssm/main.py
apex_engine/src/apexsim/models/ssm_world_model.py
apex_engine/configs/ssm_fast.yaml
```

Connection to research

The Mamba paper motivates selection because content-dependent reasoning is a weakness of fixed linear time-invariant SSMs. Its efficiency claims depend on the complete algorithm and hardware-aware implementation, not merely using a gate.

Check your understanding before continuing

What makes a state update selective?

Why is bounded retention helpful but not sufficient?

What evidence would justify a full Mamba dependency in APEX?

Solutions and reasoning

1. The current input changes parameters controlling memory propagation, writing or reading. 2. It reduces explosion risk, but the model can still forget too quickly, ignore inputs or learn useless gates. 3. Long histories where GRU/Transformer baselines fail or are too slow, sufficient data, reproducible gains by horizon, and deployment hardware that benefits from the implementation.

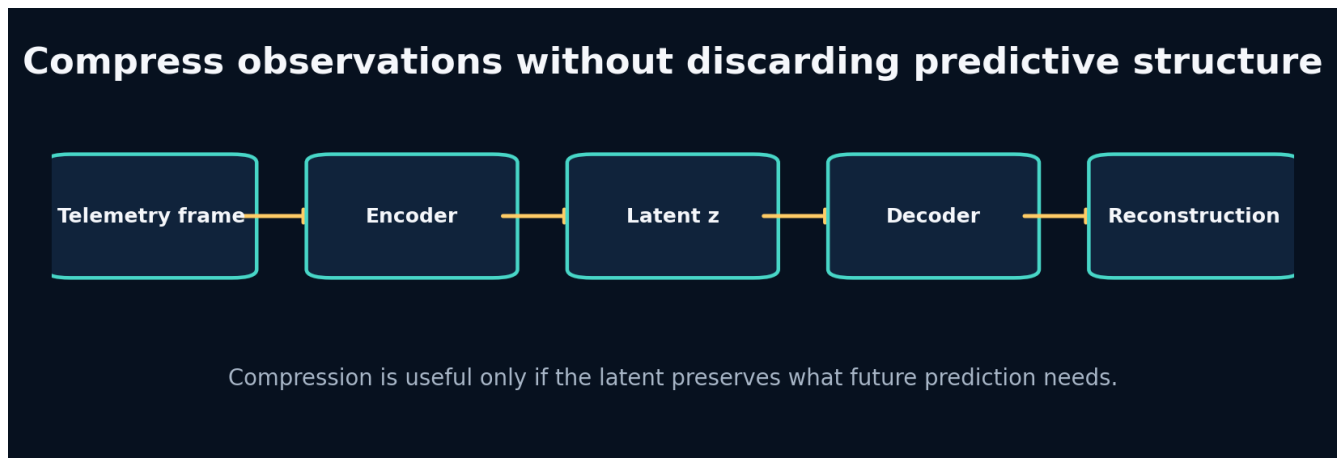
Independent build challenge

Create a dataset with rare event tokens that determine a target many steps later. Compare fixed linear SSM, selective cell and GRU while plotting learned retention around events.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

14. Autoencoders and Variational Latent State

Instructor objective: Learn representation compression by building it, then add probabilistic latent variables without confusing reconstruction with useful world modelling.



14. Autoencoders and Variational Latent State

The problem that earns this chapter

Raw observations can contain redundant or noisy dimensions. A world model may benefit from a compact latent state, but compression can discard precisely the small variable needed for future prediction. A good reconstruction is not automatically a good predictive representation.

Predict before reading

A telemetry vector has 12 features, but only speed, curvature and tyre age determine the next state. If an autoencoder compresses to three dimensions, will those three necessarily correspond to the three causal variables? Why or why not?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

An autoencoder learns an encoder $z=f(x)$ and decoder $\hat{x}=g(z)$ by minimizing reconstruction error. The bottleneck pressures the latent to preserve common information. Nothing requires each dimension to align with a human variable, and high-variance nuisance features may dominate.

A variational autoencoder predicts a distribution—usually mean and log variance—rather than one code. Sampling uses $z = \mu + \sigma \odot \epsilon$, which keeps the path differentiable with respect to μ and σ . A KL term regularizes the posterior toward a prior, enabling sampling but creating a tradeoff: too weak and the latent is irregular; too strong and it can ignore observations.

For world models, the latent should preserve information useful for dynamics and reward, not only instantaneous reconstruction. Temporal prediction losses or task probes test this.

Vocabulary that now has a job

Concept: Encoder - Meaning in plain language: Maps a high-dimensional observation into a compact representation. **- Role inside APEX:** Compresses telemetry/history into latent state.

Concept: Decoder - Meaning in plain language: Reconstructs or predicts observable state from latent representation. - **Role inside APEX:** Produces interpretable future telemetry.

Concept: Reparameterization - Meaning in plain language: Express stochastic sampling as differentiable transformation of parameter-free noise. - **Role inside APEX:** Allows RSSM/VAE posterior training.

Concept: KL divergence - Meaning in plain language: Penalty measuring difference between posterior and prior distributions. - **Role inside APEX:** Regularizes stochastic latent state and aligns imagination with inference.

Worked example: calculate it by hand

For one latent dimension with $\mu=0.2$, log variance=-1:

Variance: $\exp(-1)=0.3679$.

Standard deviation: $\exp(-0.5)=0.6065$.

If sampled noise $\epsilon=1.0$, then $z=0.2+0.6065=0.8065$.

KL to standard normal is $-0.5(1 + \log\text{var} - \mu^2 - \exp(\log\text{var}))$.

Substitute: $-0.5(1-1-0.04-0.3679)=0.20395$.

The KL is zero only when posterior mean is zero and variance is one. Driving every posterior to that point would erase observation information.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/16_vae_reparameterization

What we are about to build

Compute posterior standard deviation, sample with reparameterization, and calculate KL for a tiny batch. Pair it with the autoencoder lab to compare deterministic and stochastic codes.

Runnable implementation

```
import torch
```

```
mu=torch.tensor([[0.2,-0.4]]); logvar=torch.tensor([[1.0,0.5]])
std=torch.exp(0.5*logvar); eps=torch.randn_like(std); z=mu+std*eps
kl=-0.5*torch.sum(1+logvar-mu.pow(2)-logvar.exp(),dim=1)
print("mu",mu,"std",std,"sample",z,"KL",kl)
```

Observed output from the packaged solution

```
mu tensor([[ 0.2000, -0.4000]]) std tensor([[0.6065, 1.2840]]) sample tensor([[ -0.1015,  0.6276]]) KL
tensor([0.3583])
```

Read the important lines like English

****Code:** `std=torch.exp(0.5*logvar)` - What the line is doing: **Convert log variance into standard deviation.** - What to inspect: ****** The factor 0.5 appears because std is square root of variance.

Code: `eps=torch.randn_like(std)` - **What the line is doing:** Sample parameter-free standard normal noise. - **What to inspect:** Random seed affects the sampled latent, not posterior parameters.

****Code:** `z=mu+std*eps` - **What the line is doing:** **Create a stochastic sample with gradients flowing through μ and std .** - **What to inspect:** ****** Inspect multiple samples to see uncertainty.

Code: `kl=...` - **What the line is doing:** Measure posterior departure from the unit Gaussian prior. - **What to inspect:** Monitor KL by dimension for collapse or overuse.

State and tensor trace

```
observation x
  ↓ encoder
 $\mu(x), \log\sigma^2(x)$ 
  ↓ sample  $\epsilon \sim N(0, I)$ 
 $z = \mu + \sigma \epsilon$ 
  ↓ decoder
reconstruction  $\hat{x}$ 
```

`loss = reconstruction error +  $\beta \times \text{KL}$`

Print latent means, standard deviations and KL—not only total loss.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Set KL weight extremely high and inspect whether reconstructions become generic. Then set it to zero and sample from the prior at inference.

Diagnose from the earliest failed contract

High KL with poor reconstruction suggests excessive regularization. Near-zero KL in every dimension plus decoder independence from z suggests posterior collapse. Good reconstruction with nonsensical prior samples suggests an unstructured latent.

Repair and lock the repair with a test

Use KL warm-up/free-nats or capacity control only after measuring the failure. Add latent probes for future speed, curvature and tyre state, and compare reconstruction versus predictive usefulness.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Deterministic autoencoder - **Choose it when:** Compression and reconstruction matter; uncertainty is not required. - **Do not choose it when:** You need coherent sampling or probabilistic belief.

Implementation: VAE - **Choose it when:** A smooth sampleable latent distribution is useful. - **Do not choose it when:** KL tradeoffs add complexity without serving the downstream task.

Implementation: Predictive encoder - **Choose it when:** Future-relevant structure matters more than reconstructing every input detail. - **Do not choose it when:** You still require high-fidelity observation generation.

Implementation: No bottleneck - Choose it when: State is already small and semantically meaningful.
- Do not choose it when: High-dimensional noisy observations overwhelm dynamics learning.

APEX telemetry is already compact, so V1 can predict state directly. The RSSM uses stochastic latent state to study uncertainty and imagination, not because compression is automatically necessary.

Transfer the lesson into Project APEX

Run autoencoder and VAE labs, then inspect RSSM posterior statistics. Create a probe predicting future speed from latent state and compare it with reconstruction loss.

Repository path to inspect

```
labs/15_autoencoder/solution.py
labs/16_vae_reparameterization/solution.py
apex_engine/src/apexsim/models/rssm.py
debugging_cases/11_kl_collapse
```

Connection to research

JEPA-style approaches challenge the need to reconstruct raw observations, arguing that useful representations can be learned by predicting abstract target embeddings. This is especially relevant when raw detail is unpredictable or irrelevant.

Check your understanding before continuing

Why can low reconstruction error coexist with poor future prediction?

What is the purpose of the KL term?

Why does reparameterization enable gradients?

Solutions and reasoning

1. The encoder may preserve visually/statistically dominant current detail while discarding small causal variables needed later. 2. It regularizes the posterior toward the prior so latent space can support coherent prior sampling/imagination. 3. The random variable is expressed as a deterministic differentiable function of parameters and independent noise, so gradients pass through μ and σ .

Independent build challenge

Train an autoencoder on synthetic telemetry with nuisance noise. Compare latent probes for future speed under reconstruction-only and reconstruction-plus-future-prediction objectives.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

15. RSSMs and Dreamer: Learning to Imagine Without Observations

Instructor objective: Implement the logic of a recurrent state-space model, distinguish posterior training from prior imagination, and understand Dreamer as a system rather than a buzzword.

RSSM separates deterministic memory from stochastic uncertainty



Training uses posterior evidence; imagination must survive on the prior.

15. RSSMs and Dreamer: Learning to Imagine Without Observations

The problem that earns this chapter

A deterministic recurrent model predicts one future. Real driving contains unobserved factors, noisy evidence and multiple plausible outcomes. An RSSM maintains deterministic memory plus a stochastic latent belief. The hardest requirement is that imagination must continue when future observations are absent.

Predict before reading

During training you know the next observation; during planning you do not. Which distribution may use the observation: the prior, posterior, both or neither? What would happen if imagination secretly used posterior information?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

An RSSM commonly has deterministic state h_t and stochastic state z_t . The transition updates h_t from previous latent/action. The prior predicts a distribution over z_t from h_t . During training, an encoder provides observation evidence and a posterior refines that distribution. The decoder predicts observation/state from h_t, z_t .

The KL term teaches the prior to resemble the posterior enough that prior-only imagination remains useful. Reconstruction or prediction terms teach the latent to represent evidence. These objectives can conflict. Posterior collapse, overconfident priors and compounding uncertainty are practical failures, not footnotes.

Dreamer adds reward/continuation models and learns actor/critic behaviour from trajectories imagined by the world model. Before trusting control, APEX must establish that interventions produce realistic, calibrated rollouts.

Vocabulary that now has a job

Concept: Deterministic state h - **Meaning in plain language:** Recurrent memory summarizing past latent/action sequence. - **Role inside APEX:** Carries stable temporal context.

Concept: Prior - **Meaning in plain language:** Latent distribution predicted without current observation. - **Role inside APEX:** Used during imagination and planning.

Concept: Posterior - Meaning in plain language: Latent distribution conditioned on current observation evidence. - **Role inside APEX:** Used during training/state inference.

Concept: Imagination - Meaning in plain language: Rollout using learned prior dynamics without future observations. - **Role inside APEX:** Generates counterfactual telemetry trajectories.

Worked example: calculate it by hand

Suppose posterior is $N(\mu_q=1, \sigma_q=0.5)$ and prior is $N(\mu_p=0, \sigma_p=1)$. The one-dimensional Gaussian KL $KL(q || p)$ is

$$\log(\sigma_p/\sigma_q) + \frac{\sigma_q^2 + (\mu_q - \mu_p)^2}{2\sigma_p^2} - \frac{1}{2}.$$

Substitute:

$\log(1/0.5)=0.693$

numerator $0.25 + 1 = 1.25$

divide by 2 gives 0.625

subtract 0.5

total 0.818

The gap signals that the prior cannot yet reproduce the posterior belief. Reducing it by making both distributions uninformative would also be bad; reconstruction/prediction must remain strong.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/17_rssm_step

What we are about to build

Construct one RSSM step: update deterministic memory, compute prior statistics, then compute posterior statistics using observation embedding. Inspect all shapes.

Runnable implementation

```
import torch
from torch import nn

cell=nn.GRUCell(5,8); prior=nn.Linear(8,4); posterior=nn.Linear(8+3,4)
h=torch.zeros(2,8); prev_z=torch.zeros(2,2); action=torch.randn(2,3); obs_embed=torch.randn(2,3)
h=cell(torch.cat([prev_z,action],-1),h)
prior_stats=prior(h); post_stats=posterior(torch.cat([h,obs_embed],-1))
print("h",h.shape,"prior",prior_stats.shape,"posterior",post_stats.shape)
```

Observed output from the packaged solution

h torch.Size([2, 8]) prior torch.Size([2, 4]) posterior torch.Size([2, 4])

Read the important lines like English

Code: `h=cell(torch.cat([prev_z,action],-1),h)` - **What the line is doing:** Advance deterministic memory using previous latent and current action. - **What to inspect:** Current observation is intentionally absent from this transition.

Code: `prior_stats=prior(h)` - **What the line is doing:** Predict latent distribution using only imagined memory. - **What to inspect:** This path must work at inference with no future observation.

Code: `post_stats=posterior(torch.cat([h,obs_embed],-1))` - **What the line is doing:** Refine latent belief using current evidence during training. - **What to inspect:** Posterior information must not leak into prior-only evaluation.

State and tensor trace

training step:

```
prev z + action → h_t → prior p(z_t|h_t)
                + observation embedding → posterior q(z_t|h_t,o_t)
posterior sample + h_t → decode target
```

imagination step:

prev z + action $\rightarrow$ h_t $\rightarrow$ prior sample $\rightarrow$ decode prediction
(no observation branch)

Evaluate both posterior reconstruction and prior rollout. They answer different questions.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Use posterior samples during test rollout. Then set KL weight to zero and compare prior imagination with posterior reconstruction.

Diagnose from the earliest failed contract

If posterior metrics are good but prior rollout is poor, inspect prior-posterior KL by horizon and latent dimension. If KL is near zero but outputs ignore observations, inspect collapse.

Repair and lock the repair with a test

Separate evaluation functions for posterior reconstruction and prior imagination. Add a test that imagination accepts no future observation tensor. Tune KL with explicit diagnostics, not only total loss.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Deterministic GRU world model - **Choose it when:** One likely future and stable V1 forecasting are sufficient. - **Do not choose it when:** Uncertainty/multi-modal futures are central.

Implementation: RSSM - Choose it when: You need a learned belief state and prior imagination. - **Do not choose it when:** Data/budget cannot support stable latent training or benefits are unmeasured.

Implementation: Ensemble deterministic models - Choose it when: You want practical epistemic uncertainty with simple training. - **Do not choose it when:** Compute or storage prevents multiple models.

Implementation: Full Dreamer actor-critic - Choose it when: Dynamics, reward and continuation models are validated and control is the target. - **Do not choose it when:** The world model can still be exploited or interventions are poorly represented.

APEX includes a compact RSSM challenger and preserves its weaker short-budget result as a debugging lesson. The deterministic GRU remains V1 primary until stochastic imagination demonstrates decision value and calibration.

Transfer the lesson into Project APEX

Train GRU and RSSM under the same session split. Compare posterior reconstruction, prior horizon error, KL, physical violations and scenario sensitivity—not just total training loss.

Repository path to inspect

```
projects/08_rssm_imagination/main.py
apex_engine/src/apexsim/models/rssm.py
apex_engine/configs/rssm_fast.yaml
debugging_cases/11_kl_collapse
```

Connection to research

DreamerV3 is an integrated agent: world model, imagined trajectories, reward/continuation prediction and actor-critic learning. Copying an RSSM cell is not equivalent to reproducing Dreamer’s training system or robustness.

Check your understanding before continuing

Why must the prior exclude current observation evidence?

What does good posterior reconstruction but poor prior rollout mean?

Why might a deterministic model outperform an RSSM on APEX V1?

Solutions and reasoning

1. The future observation is unavailable during imagination; including it leaks the answer and invalidates the simulator. 2. The latent encoder can explain observed frames, but learned dynamics cannot predict the corresponding latent beliefs without evidence. 3. Small data, short training, near-deterministic telemetry, KL optimization difficulty and evaluation focused on mean trajectory can all favour the simpler model.

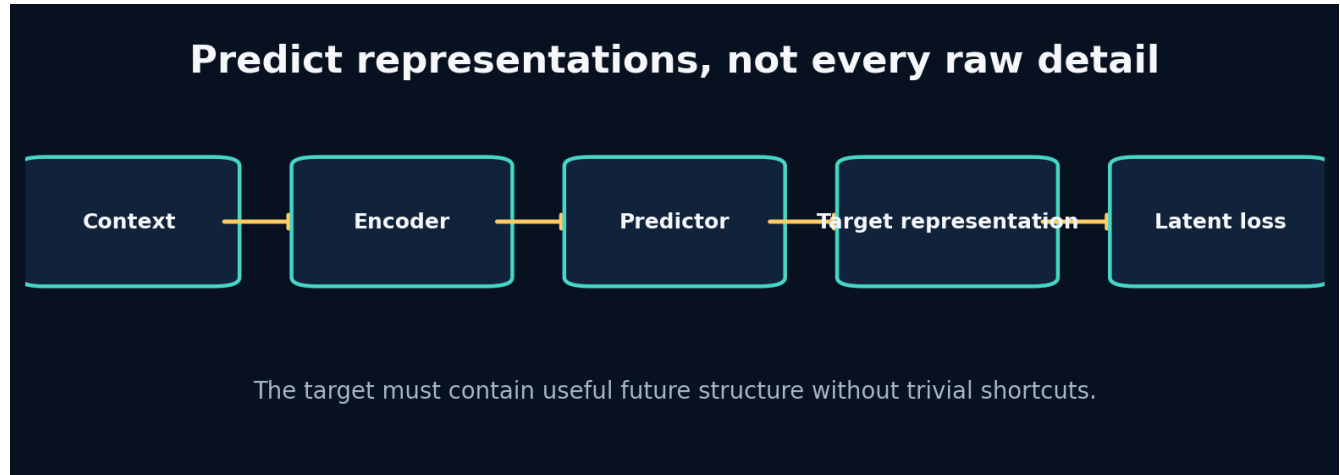
Independent build challenge

Implement a two-dimensional RSSM on a stochastic toy car. Plot posterior and prior distributions over time, then show a case where uncertainty widens under an unseen control sequence.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

16. JEPA, LeWorldModel and Fine-Tuning Existing Representations

Instructor objective: Understand predictive representation learning, decide when not to reconstruct raw observations, and build a disciplined fine-tuning strategy.



16. JEPA, LeWorldModel and Fine-Tuning Existing Representations

The problem that earns this chapter

Pixel or telemetry reconstruction forces a model to spend capacity on every observable detail, including noise that may not matter for future decisions. JEPA-style methods predict target representations instead. LeWorldModel-style work explores efficient latent world modelling and planning. The engineering question is not which acronym is newest; it is what target contains the information APEX needs.

Predict before reading

Suppose two future telemetry frames differ only in an unpredictable sensor-noise feature. A raw reconstruction loss treats them as different. Should a predictive representation loss treat them as different? What determines the answer?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

Joint-embedding predictive architectures encode context and target views, then train a predictor to infer the target representation from context. The target encoder defines what counts as meaningful similarity. Because the loss is in representation space, the model can ignore some raw variation—if the representation has learned to ignore it.

This creates a new failure possibility: representation collapse or shortcuts. Stop-gradient/target-encoder design, masking strategy and representation diagnostics matter. In telemetry, a future-latent target could emphasize dynamics while de-emphasizing sensor noise, but it might also discard rare safety-critical events.

Fine-tuning an existing model makes sense when its input semantics, sequence structure and learned invariances transfer. Adaptation choices range from a frozen encoder plus new head, through

parameter-efficient adapters, to full fine-tuning. Start with the least invasive option that can represent the domain shift, and compare against training a small model from scratch.

Vocabulary that now has a job

Concept: Representation target - Meaning in plain language: The encoded feature vector the predictor must infer. - **Role inside APEX:** Potential future-dynamics target instead of raw telemetry.

Concept: Stop gradient / target encoder - Meaning in plain language: Prevents both sides from trivially co-adapting in unstable ways. - **Role inside APEX:** Part of a non-collapsing predictive objective.

Concept: Linear probe - Meaning in plain language: A simple supervised head used to test what information a representation contains. - **Role inside APEX:** Measures speed, curvature, tyre or event information in latent state.

Concept: Fine-tuning - Meaning in plain language: Adapt pretrained parameters to a new domain/task. - **Role inside APEX:** Possible when a relevant sequence/world model checkpoint exists.

Worked example: calculate it by hand

Compare two losses for target vector $y=[10.0, 0.02]$ where the second feature is pure noise. Prediction A is $[9.8, 0.50]$; prediction B is $[9.8, -0.40]$. Raw MSE differs because of noise.

If a target encoder maps both raw targets to a dynamics representation $[\text{speed_bin}=10, \text{regime}=\text{straight}]$, their representation loss can be identical. This is desirable only if the discarded noise truly has no predictive or decision value.

For fine-tuning, suppose a pretrained encoder has one million parameters. A frozen-head experiment trains 5,000 parameters; an adapter trains 50,000; full tuning changes all million. These are nested hypotheses about how much the representation must change.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/15_autoencoder

What we are about to build

Use the autoencoder lab as a concrete representation baseline, then modify its objective conceptually: predict a future latent code rather than reconstruct the current input. Add a linear probe for a known future variable.

Runnable implementation

```
import torch
from torch import nn

torch.manual_seed(3); x=torch.randn(256,12)
enc=nn.Linear(12,3); dec=nn.Linear(3,12); opt=torch.optim.Adam(list(enc.parameters())
+list(dec.parameters()),lr=0.03)
for _ in range(80):
    opt.zero_grad(); z=torch.tanh(enc(x)); recon=dec(z); loss=((recon-x)**2).mean(); loss.backward();
    opt.step()
print("latent",z.shape,"loss",float(loss))
```

Observed output from the packaged solution

```
latent torch.Size([256, 3]) loss 0.7505914568901062
/mnt/data/Project_APEX_Engineering_Apprenticeship/labs/15_autoencoder/solution.py:8: UserWarning:
Converting a tensor with requires_grad=True to a scalar may lead to unexpected behavior.
Consider using tensor.detach() first. (Triggered internally at
/pytorch/torch/csrc/autograd/generated/python_variable_methods.cpp:836.)
print("latent",z.shape,"loss",float(loss))
```

Read the important lines like English

Code: `z=torch.tanh(enc(x))` - What the line is doing: Produce a learned representation through a bottleneck. - **What to inspect:** Inspect variance and pairwise similarity to detect collapse.

Code: `recon=dec(z)` - What the line is doing: The original lab asks the latent to reconstruct raw input. - **What to inspect:** Replace/augment this with future-target representation prediction.

Code: `loss=((recon-x)2).mean()` - What the line is doing:** Raw reconstruction values every feature according to scale and frequency. - **What to inspect:** Normalize or weight features, or choose a different target.

State and tensor trace

```
context history → context encoder → predictor → predicted target embedding
future target → target encoder → target embedding
                        ↓
                    latent-space loss
```

After training, freeze the encoder and probe whether latent state predicts physical variables and future events.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Train both context and target encoders with no asymmetry or variance checks until they output nearly constant vectors. Observe low latent loss and useless probes.

Diagnose from the earliest failed contract

Track per-dimension variance, embedding covariance, nearest-neighbour diversity and probe accuracy. Low loss with collapsed variance is not successful representation learning.

Repair and lock the repair with a test

Use an appropriate target/stop-gradient design, masking that prevents shortcuts, normalization, and diagnostic probes. Add rare-event retrieval tests so abstraction does not erase critical evidence.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Raw reconstruction - Choose it when: Exact observation generation or interpretable state recovery matters. - **Do not choose it when:** Unpredictable detail dominates and decisions depend on abstract dynamics.

Implementation: Predictive latent loss - Choose it when: Future-relevant representation is the main goal. - **Do not choose it when:** You cannot validate what information the latent discarded.

Implementation: Frozen pretrained encoder - Choose it when: Source and target semantics align and data are limited. - **Do not choose it when:** The representation lacks essential F1 variables.

Implementation: Full fine-tuning - Choose it when: You have sufficient data/compute and major domain adaptation is needed. - **Do not choose it when:** Small data risks catastrophic overfitting or a small scratch model is already competitive.

APEX V1 trains small models from scratch because the canonical telemetry state is compact and no assumed checkpoint perfectly matches its action/context semantics. The curriculum nevertheless prepares an adapter/probe-first fine-tuning experiment for future larger datasets.

Transfer the lesson into Project APEX

Add a latent probe suite to GRU, SSM and RSSM hidden states. If using a pretrained sequence encoder, freeze it first, train the same prediction head, and compare data efficiency and horizon curves.

Repository path to inspect

```
labs/15_autoencoder/solution.py
apex_engine/src/apexsim/models/
apex_engine/src/apexsim/evaluation.py
research_reading/
```

Connection to research

I-JEPA predicts representations of target image regions from context rather than reconstructing pixels. LeWorldModel explores latent world modelling and planning efficiency. Transfer their design principles only after mapping observation, action, target and evaluation semantics to the F1 domain.

Check your understanding before continuing

Why can latent loss be low under representation collapse?

What should a linear probe tell you?

What is the safest first fine-tuning experiment?

Solutions and reasoning

1. If both target and prediction are nearly constant, they match without containing information. 2. Whether a simple readout can recover a specified physical or predictive variable, revealing what information is accessible in the representation. 3. Freeze the pretrained backbone, train a small task head under the same split, and compare with simple baselines before unfreezing parameters.

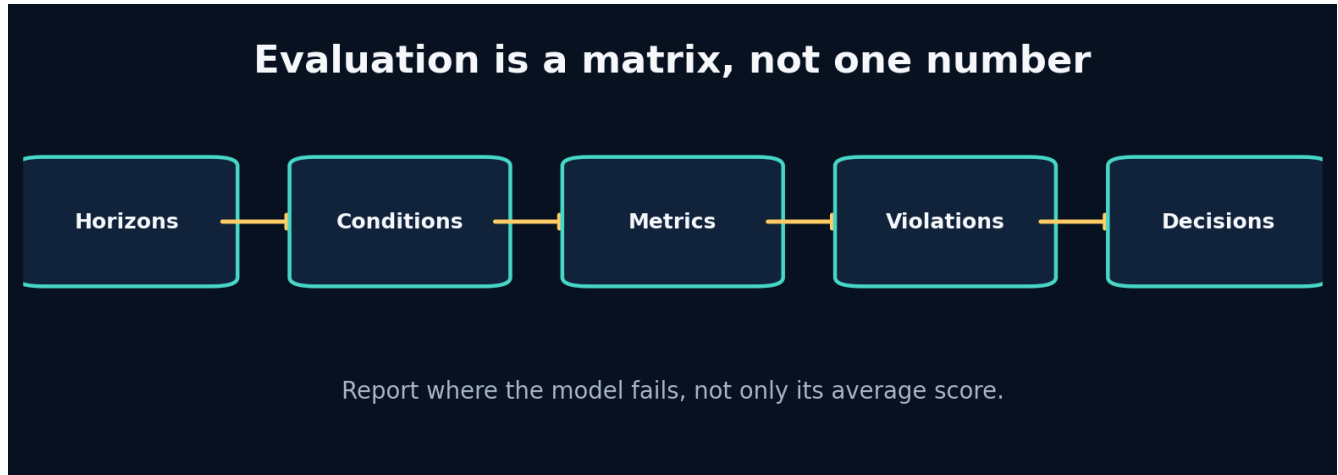
Independent build challenge

Build a future-latent predictor on synthetic telemetry. Evaluate reconstruction, future-state probes, rare-event retrieval and rollout value. Write a decision record comparing it with an RSSM objective.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

17. Evaluation, Ablation and Out-of-Distribution Testing

Instructor objective: Build an evaluation matrix that reveals where a world model works, why it works, and when it should refuse to be trusted.



17. Evaluation, Ablation and Out-of-Distribution Testing

The problem that earns this chapter

A single average MAE can hide catastrophic braking failures, worsening long-horizon error, impossible speeds, or performance limited to one track. A simulation engine needs evidence across horizons, features, conditions and interventions.

Predict before reading

Model A has lower average speed MAE than Model B, but A predicts negative speed in 2% of wet-braking rollouts while B never does. Which model is better? State the missing operational information required to decide.

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

Evaluation begins with the decision. If a race engineer uses the simulator to compare a six-second throttle scenario, horizon-six causal sensitivity and physical validity matter more than one-step average fit. Metrics are measurements of requirements, not a scoreboard detached from use.

Ablation removes or changes one information source while holding other factors fixed. It answers causal questions about the implementation: Does weather context help? Does VH-like second channel—here, perhaps curvature or tyre age—matter? Randomly changing many settings answers nothing cleanly.

Out-of-distribution tests define shifts deliberately: unseen track, heavier rain, longer tyre age, unusual controls, different sample rate. Failure under OOD is expected; the goal is to detect and quantify it, then decide whether to abstain, retrain or constrain use.

Vocabulary that now has a job

Concept: Horizon metric - Meaning in plain language: Error measured separately at each future step.

- **Role inside APEX:** Defines useful simulation duration.

Concept: Guardrail metric - Meaning in plain language: A metric that must remain within a safety/validity limit even if primary error improves. - **Role inside APEX:** Negative speed, progress bounds and control sensitivity.

Concept: Ablation - Meaning in plain language: A controlled removal or replacement testing one hypothesis. - **Role inside APEX:** State-only, no weather, no geometry, model-family comparisons.

Concept: OOD test - Meaning in plain language: Evaluation under a deliberately shifted distribution. -

Role inside APEX: Unseen circuits, weather or control regimes.

Worked example: calculate it by hand

Suppose Model A and B have:

Metric

A

B

1-step MAE

0.4

0.5

20-step MAE

8.0

5.0

Negative-speed rate

2%

0%

Scenario latency

30 ms

12 ms

If the UI serves 20-step scenarios, B is stronger despite worse one-step fit. If only one-step filtering is needed, A may be acceptable if violations are constrained. The “best model” is a vector of requirement tradeoffs plus thresholds, not one scalar.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/09_loss_comparison

What we are about to build

Compare loss functions on controlled predictions, then connect optimization loss to evaluation metrics. The loss trains parameters; the metric judges the product and need not be identical.

Runnable implementation

```
import torch
from torch.nn import functional as F

pred=torch.tensor([0.,1.,2.,20.]); target=torch.tensor([0.,1.,2.,3.])
print("MSE",F.mse_loss(pred,target).item())
print("MAE",F.l1_loss(pred,target).item())
print("Huber",F.huber_loss(pred,target).item())
```

Observed output from the packaged solution

MSE 72.25
MAE 4.25
Huber 4.125

Read the important lines like English

Code: mse = ... - **What the line is doing:** Penalize squared residuals, emphasizing large errors. - **What to inspect:** Sensitive to scale and outliers.

Code: mae = ... - **What the line is doing:** Measure average absolute physical error. - **What to inspect:** More robust but less smooth at zero.

Code: compare losses - **What the line is doing:** Reveal how optimization priorities differ on the same predictions. - **What to inspect:** Do not select by training loss alone.

State and tensor trace

```
run artifact
├─ overall metrics
├─ horizon curves
├─ per-feature errors
├─ subgroup tables
├─ physical violations
├─ intervention sensitivity
└─ baseline-relative improvement
```

Every plot should include sample count and split identity.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Average all horizons and all conditions into one MAE. Then remove weather from inputs but accidentally change random seed and training epochs too.

Diagnose from the earliest failed contract

Ask whether compared runs share dataset hash, split, normalizer, seed set, budget and evaluation code. If multiple variables changed, the result is not an ablation.

Repair and lock the repair with a test

Store experiment manifests and compare matched runs. Add confidence intervals across seeds/sessions where feasible. Make promotion gates explicit: primary metric, guardrails, latency and artifact completeness.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: MAE/RMSE - Choose it when: Continuous state error in interpretable units. - **Do not choose it when:** They are the only evidence for a simulator.

Implementation: Horizon curves - Choose it when: Autoregressive use or planning matters. - **Do not choose it when:** Never; at least inspect them for rollouts.

Implementation: Physical penalties - Choose it when: Known invalid regions can be expressed and monitored. - **Do not choose it when:** They replace empirical outcome evaluation.

Implementation: Human scenario review - Choose it when: Qualitative causal behaviour and UI trust matter. - **Do not choose it when:** It substitutes for reproducible quantitative tests.

APEX reports baseline-relative error by horizon, per-feature metrics, physical violations, scenario sensitivity and run metadata. Model promotion is a documented decision, not automatic best-MAE selection.

Transfer the lesson into Project APEX

Run the GRU, SSM and RSSM references. Build one comparison report with matched data and budgets, then explain why the simplest winning model remains primary.

Repository path to inspect

```
apex_engine/src/apexsim/evaluation.py
apex_engine/src/apexsim/pipeline/stages.py
apex_engine/artifacts/runs/
field_workbook/
```

Connection to research

World-model papers choose benchmarks and metrics that reflect their claims. When adapting a paper to F1, reproduce the mechanism but redefine evaluation around telemetry dynamics, interventions, uncertainty and deployment conditions.

Check your understanding before continuing

What makes an experiment a true ablation?

Why should error be normalized as well as reported in physical units?

What is an abstention policy?

Solutions and reasoning

1. One targeted component changes while data, split, budget, evaluation and other variables remain fixed. 2. Physical units are interpretable; normalized error helps compare features with different scales. Both are useful. 3. A rule that withholds or flags predictions when evidence is outside trusted conditions or uncertainty/violations exceed thresholds.

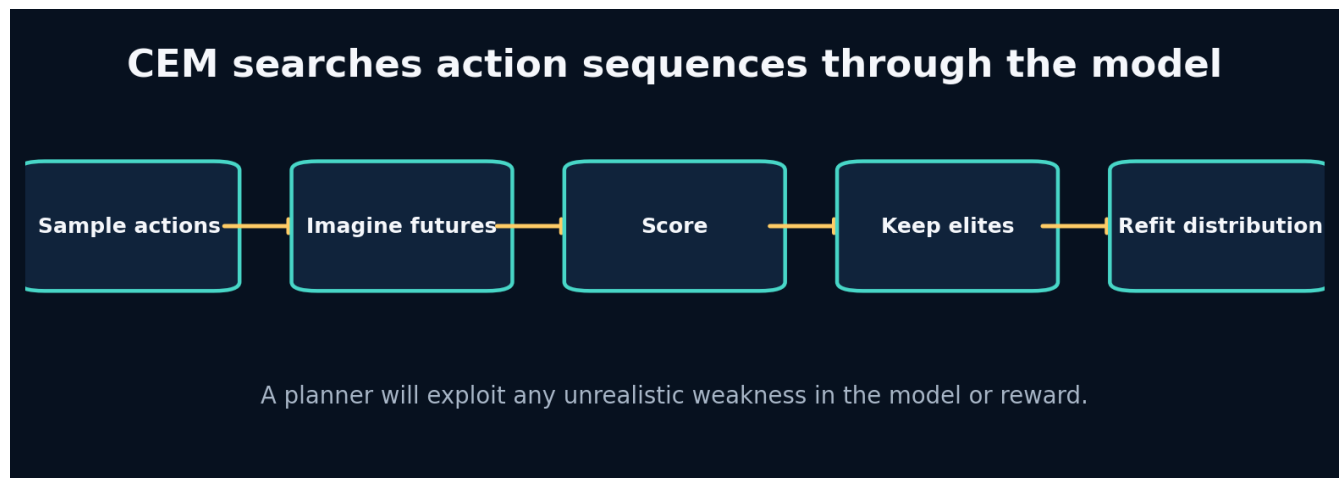
Independent build challenge

Design a complete APEX evaluation matrix across three horizons, three tracks, wet/dry conditions and five metrics. Define promotion and abstention criteria before seeing results.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

18. Planning With MPC and the Cross-Entropy Method

Instructor objective: Use a world model to compare action sequences, understand CEM search, and learn why planners exploit model and reward defects.



18. Planning With MPC and the Cross-Entropy Method

The problem that earns this chapter

Prediction answers “what happens if we do this?” Planning asks “what should we do?” A planner proposes action sequences, imagines outcomes through the world model, scores them, and improves its proposal. This creates a powerful adversary: it actively searches for unrealistic trajectories that maximize the score.

Predict before reading

A reward gives +1 for speed and no penalty for leaving the track or braking instability. What action sequence will a planner prefer? Why is this not primarily a planner bug?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

Model predictive control repeatedly plans over a finite horizon, executes only the first action, observes the new state, and replans. This feedback limits drift compared with committing to a long open-loop sequence.

The cross-entropy method maintains a distribution over action sequences. It samples candidates, rolls them through the model, keeps an elite fraction, and refits mean/variance toward those elites. Repetition concentrates search around high-scoring actions.

Planner quality is bounded by world-model validity and reward completeness. Optimization pressure finds loopholes humans do not notice in average evaluation. Therefore planning tests should include adversarial actions, uncertainty penalties, constraints and real-environment verification before control is trusted.

Vocabulary that now has a job

Concept: Model predictive control - Meaning in plain language: Plan over a horizon, execute a short prefix, observe, and replan. - **Role inside APEX:** Future scenario/strategy layer after dynamics validation.

Concept: CEM - Meaning in plain language: Sampling optimizer that iteratively refits an action distribution to elite candidates. - **Role inside APEX:** Simple derivative-free planner through imagined telemetry.

Concept: Reward model - Meaning in plain language: Function scoring predicted trajectories for the decision objective. - **Role inside APEX:** Could combine progress, stability, tyre use and constraint penalties.

Concept: Model exploitation - Meaning in plain language: Planner finds actions that score well because the learned model is wrong. - **Role inside APEX:** A major risk under unusual controls.

Worked example: calculate it by hand

Suppose CEM samples 500 six-step throttle sequences. It scores final-speed error to a target plus smoothness. It keeps 50 elites (top 10%). New mean and standard deviation are the elite sample statistics.

If initial mean is 0.5 and standard deviation 0.3, one iteration may shift the early-step means toward 0.8 where acceleration helps. After several iterations variance shrinks. Too-rapid variance collapse can trap search; a minimum standard deviation preserves exploration.

MPC then applies only the first action, receives new evidence and replans. The remaining five actions are proposals, not a fixed command commitment.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/18_cem_planning

What we are about to build

Run CEM against a tiny differentiable-free speed simulator. Watch mean action sequence and best score improve over five iterations.

Runnable implementation

import numpy **as** np

```
rng=np.random.default_rng(0); horizon=6; mean=np.full(horizon,0.5); std=np.full(horizon,0.3)
for it in range(5):
    actions=np.clip(rng.normal(mean,std,(500,horizon)),0,1)
    speed=40+np.cumsum(2*actions-0.4,axis=1)
    score=-(speed[:,1]-48)**2-0.1*np.sum(np.diff(actions,axis=1)**2,axis=1)
    elite=actions[np.argsort(score)[-50:]]
    mean,std=elite.mean(0),elite.std(0)+1e-3
    print(it,mean.round(2),score.max().round(3))
```

Observed output from the packaged solution

```
0 [0.71 0.68 0.7 0.62 0.67 0.71] -1.123
1 [0.84 0.8 0.82 0.82 0.83 0.82] -0.019
2 [0.86 0.84 0.86 0.88 0.87 0.9 ] -0.005
3 [0.87 0.85 0.85 0.87 0.89 0.87] -0.001
4 [0.86 0.84 0.85 0.88 0.89 0.87] -0.001
```

Read the important lines like English

Code: `actions=np.clip(rng.normal(mean,std,(500,horizon)),0,1)` - **What the line is doing:** Sample bounded candidate action sequences from the current proposal. - **What to inspect:** Clipping changes the distribution near limits.

Code: `score=...` - **What the line is doing:** Evaluate imagined outcome and smoothness under the toy world model. - **What to inspect:** Every omitted constraint becomes a possible exploit.

Code: `elite=actions[np.argsort(score)[-50:]]` - **What the line is doing:** Select the highest-scoring candidate sequences. - **What to inspect:** Elite fraction controls search pressure.

Code: `mean,std=elite.mean(0),elite.std(0)+1e-3` - **What the line is doing:** Refit proposal distribution and preserve minimum exploration. - **What to inspect:** Monitor premature variance collapse.

State and tensor trace

proposal distribution over action sequences

↓ sample N candidates

world model rollout for each candidate

↓ reward + constraints

select top K elites

↓ fit new mean/std

repeat → execute first action → observe → replan

Log the best candidate, elite diversity, uncertainty and physical violations each iteration.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Remove the smoothness term and allow throttle beyond one. Add a model region where extreme throttle incorrectly increases speed without penalty. Watch CEM discover it.

Diagnose from the earliest failed contract

Replay planned trajectories under a trusted simulator or held-out real transitions. Compare planner actions with the training action distribution. Large OOD actions and low predicted uncertainty are red flags.

Repair and lock the repair with a test

Enforce hard action bounds, add physically justified constraints, penalize uncertainty/OOD distance, and use receding-horizon verification. Create adversarial planner tests as part of model evaluation.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Grid search - Choose it when: Action space and horizon are tiny. - **Do not choose it when:** Combinatorial sequences.

Implementation: CEM - Choose it when: Continuous bounded actions, no gradients required and batch rollout is cheap. - **Do not choose it when:** Model evaluations are extremely expensive or multi-modal search collapses.

Implementation: Gradient planning - Choose it when: World model/reward are differentiable and smooth. - **Do not choose it when:** Discrete actions, poor local optima or unstable gradients.

Implementation: Actor network - Choose it when: Fast repeated decisions justify amortizing planning through policy learning. - **Do not choose it when:** World model/reward is not validated or policy can exploit it unseen.

APEX V1 exposes manual counterfactual controls and includes a latent MPC project, but does not claim autonomous race control. Planning becomes a later stage after dynamics and reward validation.

Transfer the lesson into Project APEX

Connect CEM to the scenario rollout interface using bounded throttle/brake actions. Evaluate planned sequences against the synthetic ground-truth environment before any live integration.

Repository path to inspect

```
labs/18_cem_planning/solution.py
projects/09_latent_mpc/main.py
apex_engine/src/apexsim/simulation.py
debugging_cases/12_planner_exploitation
```

Connection to research

Dreamer replaces repeated online search with an actor and critic trained from imagined trajectories, but the exploitation problem remains. Better optimization can make model defects more dangerous, not less.

Check your understanding before continuing

Why does MPC execute only the first planned action?

What does elite variance tell you?

How can uncertainty be used in planning?

Solutions and reasoning

1. New observations correct model drift and changing conditions before the next action. 2. How concentrated high-scoring candidates are; rapid collapse can indicate convergence or premature loss of exploration. 3. Penalize uncertain trajectories, constrain planning to trusted regions, or trigger abstention/human review.

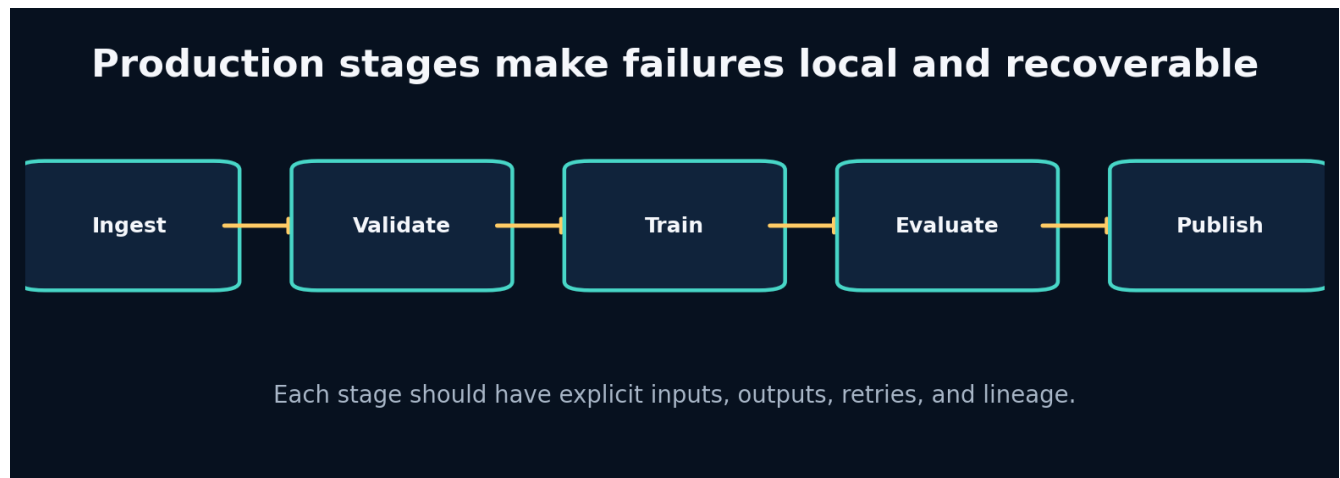
Independent build challenge

Add braking and curvature constraints to the CEM lab. Create a hidden model defect, show the planner exploit it, then design an evaluation that catches the exploit before deployment.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

19. Production Pipelines, Airflow, Registry and Monitoring

Instructor objective: Refactor experimental code into idempotent stages with observable lineage, then understand when orchestration becomes necessary.



19. Production Pipelines, Airflow, Registry and Monitoring

The problem that earns this chapter

A notebook can ingest, train and plot once. A production system must retry failed downloads, validate versions, reproduce models, backfill sessions, avoid overwriting artifacts, and tell operators which stage failed. Orchestration should coordinate trustworthy functions, not rescue tangled code.

Predict before reading

A pipeline fails after training but before publication. On retry, should it retrain from scratch, reuse the checkpoint, or overwrite the run directory? What information is needed to choose safely?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

A stage has explicit inputs, outputs, configuration, failure modes and idempotency policy. Ingestion writes canonical data; validation writes a report; training writes a checkpoint; evaluation writes metrics; publication writes UI/API products. Each artifact is immutable within a run and linked by identifiers/hashes.

Airflow schedules and tracks task dependencies, retries and backfills. Business logic should remain in ordinary tested Python functions so the same stage runs locally, in tests and under orchestration. Passing large arrays through orchestration metadata is a smell; pass durable artifact references.

A registry records runs, configurations, dataset versions, metrics and status. Monitoring then covers pipeline health, data drift, model behaviour and product delivery. Logging without stable run IDs and structured fields is not lineage.

Vocabulary that now has a job

Concept: Idempotency - Meaning in plain language: Repeating a stage with the same inputs yields the same result or safely reuses it. - **Role inside APEX:** Enables retries and backfills.

Concept: Artifact lineage - Meaning in plain language: Trace which data, code and configuration produced an output. - **Role inside APEX:** Every UI prediction links to model run and dataset.

Concept: Orchestrator - Meaning in plain language: Coordinates when stages run and how failures/retries are managed. - **Role inside APEX:** Airflow DAG wraps tested stage functions.

Concept: Registry - Meaning in plain language: Queryable record of runs, statuses, metrics and artifact locations. - **Role inside APEX:** SQLite V1, expandable to a service/database.

Worked example: calculate it by hand

Assume training checkpoint hash abc123 exists and evaluation failed because disk was temporarily full.

A safe retry checks:

Same run ID and immutable configuration hash.

Same dataset/normalizer hashes.

Checkpoint exists, is complete and matches metadata.

Training stage status is succeeded.

Evaluation output is absent or marked failed.

Then retry evaluation only. If configuration changed, create a new run rather than silently reusing the checkpoint. If checkpoint integrity is uncertain, retrain in a new or repaired stage path.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/20_pipeline_contract

What we are about to build

Use the miniature stage artifact writer to learn explicit stage boundaries, then trace the production APEX runner and Airflow DAG that call the same business functions.

Runnable implementation

```
from pathlib import Path
import json
```

```
run=Path('artifacts/education_pipeline'); run.mkdir(parents=True,exist_ok=True)
stages=['ingest','validate','window','train','evaluate','publish']
for i,name in enumerate(stages):
    path=run/f'{i:02d}_{name}.json'; path.write_text(json.dumps({'stage':name,'status':'succeeded'}))
    print(path)
```

Observed output from the packaged solution

```
artifacts/education_pipeline/00_ingest.json
artifacts/education_pipeline/01_validate.json
artifacts/education_pipeline/02_window.json
artifacts/education_pipeline/03_train.json
artifacts/education_pipeline/04_evaluate.json
artifacts/education_pipeline/05_publish.json
```

Read the important lines like English

Code: `run.mkdir(parents=True,exist_ok=True)` - **What the line is doing:** Create a stable run namespace for artifacts. - **What to inspect:** A production run directory should not mix different configuration hashes.

Code: `path.write_text(json.dumps(...))` - **What the line is doing:** Persist machine-readable stage evidence. - **What to inspect:** Use atomic temporary-file replacement for critical artifacts.

Code: `stages=[...]` - **What the line is doing:** Make dependencies visible and independently testable. - **What to inspect:** Do not create one giant “do everything” task.

State and tensor trace

CLI / Airflow / test

↓

ordinary Python stage function

↓

durable artifact + metadata

↓

registry status transition

↓

next stage receives artifact path

A retry should start from the last valid durable boundary.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Write every run to latest/, pass a large tensor through Airflow XCom, and combine ingestion/training/publication in one task. Simulate a publication failure.

Diagnose from the earliest failed contract

Look for overwritten evidence, ambiguous status and work that must repeat unnecessarily. The earliest failed design is usually the artifact/stage contract, not the scheduler.

Repair and lock the repair with a test

Use immutable run IDs, atomic writes, explicit stage manifests and small task messages containing paths/IDs. Add pipeline tests that run twice and assert safe reuse or deterministic replacement.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Local Python runner - Choose it when: Development, CI and small reproducible workflows. - **Do not choose it when:** Complex schedules, backfills and multi-worker operations.

Implementation: Airflow - Choose it when: Batch dependencies, retries, schedules and historical backfills matter. - **Do not choose it when:** You need low-latency event streaming or have only one trivial script.

Implementation: Experiment tracker - Choose it when: Comparing many model runs and artifacts. - **Do not choose it when:** You assume it replaces data/version contracts and orchestration.

Implementation: Model registry - Choose it when: Promotion, rollback and serving require a governed model identity. - **Do not choose it when:** A notebook checkpoint with no deployment lifecycle.

APEX implements a local runner as the source of truth, a thin Airflow DAG, a run registry and immutable run artifacts. The full Airflow service is an optional deployment layer, not required for learning core models.

Transfer the lesson into Project APEX

Run the same configuration twice under different IDs. Compare manifests, then deliberately fail publication and resume from the last successful stage. Inspect API/UI links back to the run directory.

Repository path to inspect

```
apex_engine/src/apexsim/pipeline/stages.py
apex_engine/src/apexsim/pipeline/runner.py
apex_engine/src/apexsim/registry.py
apex_engine/dags/apexsim_dag.py
debugging_cases/13_artifact_overwrite
debugging_cases/14_airflow_payload
```

Connection to research

Reproducible world-model research depends on production habits: immutable datasets, exact split policies, seeds, configuration, checkpoints and evaluation code. Without them, architecture comparisons are anecdotes.

Check your understanding before continuing

Why should Airflow tasks pass paths instead of arrays?

What makes a stage idempotent?

What four layers should monitoring cover?

Solutions and reasoning

1. Durable storage survives retries/workers, avoids metadata-size limits and preserves lineage. 2. Same logical inputs/config produce the same content or a safely detected reusable artifact without duplicating side effects. 3. Pipeline execution, input data/contract, model behaviour/drift and delivered product/API/UI health.

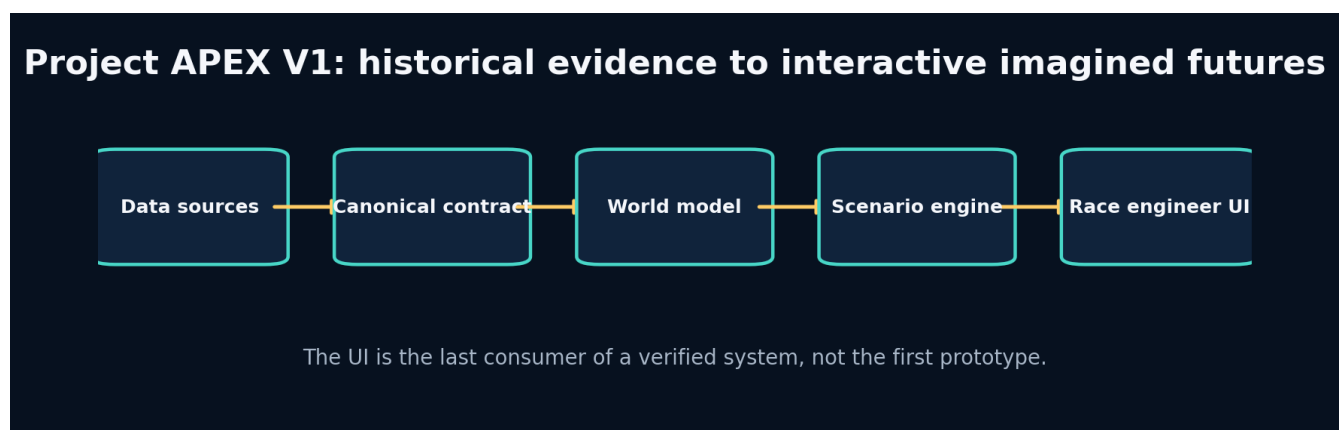
Independent build challenge

Create a two-run backfill that ingests two sessions, trains one model per session and publishes reports. Kill the process mid-run, restart it, and prove no successful work is corrupted or needlessly repeated.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

20. Build Project APEX V1 End to End—and Then Outgrow It

Instructor objective: Assemble the complete system, run a verified world-model simulation, use the UI responsibly, and define the research path toward F1 25 and an original architecture.



20. Build Project APEX V1 End to End—and Then Outgrow It

The problem that earns this chapter

The final challenge is integration. Correct individual pieces can still form a wrong system: adapters may disagree with windows, normalizers may differ between training and UI, scenario controls may overwrite state, or a beautiful interface may present false precision. We now build one traceable path from evidence to an imagined future.

Predict before reading

Before running anything, list the artifacts that must exist for a UI scenario to be reproducible: data, split, normalizer, model, evaluation, scenario inputs and software identity. Which of these should the UI be allowed to modify?

Write your answer in the Field Workbook. A prediction is valuable even when it is wrong, because it exposes the mental model we need to repair.

Build the intuition from the system outward

APEX V1 is a historical telemetry world-simulation engine. It does not claim complete car physics or autonomous strategy. It answers a narrower, testable question: given recent canonical telemetry and an explicit sequence of future controls/context, can a learned transition model generate plausible short-horizon future state?

The system begins with synthetic data so every causal rule is inspectable and offline tests always run. It accepts FastF1/OpenF1 canonical data when available. A deterministic GRU is primary; an SSM-style model and RSSM are challengers. Evaluation determines the trusted horizon. The scenario engine modifies only designated future controls/context. The UI displays recorded and imagined trajectories with model/run identity and limitations.

The next stages are research, not feature accumulation: improve state observability, calibrate uncertainty, add track/generalization splits, ingest F1 25 UDP, learn richer latent dynamics, and validate planning. An original architecture should be proposed only after a measured failure cannot be fixed by data, objective or simpler model choices.

Vocabulary that now has a job

Concept: Scenario contract - Meaning in plain language: Exactly which future controls/context may be changed and how. - **Role inside APEX:** Throttle, brake, rain/grip and tyre assumptions under bounded transformations.

Concept: Trusted horizon - Meaning in plain language: Maximum forecast duration meeting error and guardrail criteria. - **Role inside APEX:** Displayed with every scenario rather than claiming unlimited simulation.

Concept: Product provenance - Meaning in plain language: User-visible link from prediction to run/model/data/config. - **Role inside APEX:** Prevents anonymous, irreproducible UI output.

Concept: Research hypothesis - Meaning in plain language: A falsifiable explanation for a measured failure and proposed change. - **Role inside APEX:** The basis for new world-model architecture work.

Worked example: calculate it by hand

A UI scenario uses history length 32 at 5 Hz (6.4 seconds) and predicts 8 frames (1.6 seconds). Suppose speed MAE by horizon is [1.0,1.3,1.8,2.5,3.4,4.6,6.2,8.5] km/h, with a guardrail of MAE ≤ 5 km/h and zero

physical violations. The trusted horizon is step 6, or 1.2 seconds, even though the model emits 1.6 seconds.

This is not failure. The system should shade or label steps 7–8 as exploratory, refuse high-stakes conclusions, and direct research toward the compounding error observed after step 6.

Do not skip the arithmetic. When a later tensor produces a surprising value, this hand calculation is the ground truth you can return to.

Guided lab

Lab folder: labs/20_pipeline_contract

What we are about to build

Run the production APEX fast configuration, inspect every artifact, launch the UI, modify one scenario control, and trace the resulting prediction back through normalizer, model and run metadata.

Runnable implementation

```
from pathlib import Path
import json
```

```
run=Path('artifacts/education_pipeline'); run.mkdir(parents=True,exist_ok=True)
stages=['ingest','validate','window','train','evaluate','publish']
for i,name in enumerate(stages):
    path=run/f'{i:02d}_{name}.json'; path.write_text(json.dumps({'stage':name,'status':'succeeded'}))
    print(path)
```

Observed output from the packaged solution

```
artifacts/education_pipeline/00_ingest.json
artifacts/education_pipeline/01_validate.json
artifacts/education_pipeline/02_window.json
artifacts/education_pipeline/03_train.json
artifacts/education_pipeline/04_evaluate.json
artifacts/education_pipeline/05_publish.json
```

Read the important lines like English

Code: `apexsim run --config configs/fast.yaml --run-id ...` - **What the line is doing:** Execute the complete source-independent pipeline. - **What to inspect:** Use a new immutable run ID and inspect stage logs.

Code: `apexsim ui --run-dir ...` - **What the line is doing:** Launch the race-engineering interface from verified artifacts. - **What to inspect:** The UI must load the same normalizer/model/config used in evaluation.

Code: `apexsim run-canonical --input-path ...` - **What the line is doing:** Feed adapter-produced canonical data through the identical pipeline. - **What to inspect:** Validate source and split semantics before comparing runs.

State and tensor trace

```
FastF1 / OpenF1 / synthetic / future UDP
    ↓ canonical contract + quality report
session split → windows → train-only normalizer
    ↓
```

GRU / SSM / RSSM checkpoint

↓ matched evaluation + trusted horizon

scenario controls → autoregressive rollout

↓

artifact bundle → registry → API/UI

For one UI point, record: source frame indices, normalized input, scenario transform, model output, inverse transform, plotted value and run ID.

The rule is simple: never accept a tensor merely because the code runs. Name every axis, check every unit, and connect every number to a physical or statistical meaning.

Break it on purpose

Load a checkpoint with the wrong normalizer, let the UI modify future target speed directly, hide the model identity, or display more decimal precision than validation supports.

Diagnose from the earliest failed contract

Reproduce the scenario outside the UI using the same artifact paths. Compare canonical input, normalized tensor and raw model output. If CLI and UI differ, the serving boundary is broken; if both fail, move earlier.

Repair and lock the repair with a test

Bundle model, normalizer, feature order and config; validate hashes on load. Make scenario fields allow-listed. Display uncertainty/trusted horizon and provenance. Add end-to-end tests comparing UI callback output with direct simulation.

A fix without a regression test is only a temporary memory of the bug.

Choosing between implementations

Implementation: Gradio UI - Choose it when: Rapid educational interactive scenarios with Python integration. - **Do not choose it when:** You require a highly customized production frontend at scale.

Implementation: FastAPI + web frontend - Choose it when: Stable service contracts, multiple clients and custom UX. - **Do not choose it when:** The API/model contract is still changing daily.

Implementation: Synthetic-first - Choose it when: Offline causality, testing and controlled failure injection. - **Do not choose it when:** You confuse synthetic accuracy with real F1 fidelity.

Implementation: F1 25 UDP migration - Choose it when: Game packets are available and canonical mapping/latency tests are ready. - **Do not choose it when:** You let packet-specific fields leak through the entire core architecture.

The delivered V1 is a complete learning and research platform, not the final Dreamer-for-F1 system. Its value is that every future architecture competes inside a tested data, rollout, evaluation and product framework.

Transfer the lesson into Project APEX

Follow START_HERE.md, run tests, execute a fast reference, launch the UI, complete Projects 1–9, then rebuild one core module without reading the solution. Use the source companion only after your own attempt.

Repository path to inspect

apex_engine/START_HERE.md
 apex_engine/src/apexsim/
 apex_engine/notebooks/
 apex_engine/tests/
 projects/10_project_apex/
 source_code_companion/

Connection to research

The path toward a Dreamer-like F1 system is staged: validated dynamics → probabilistic belief/calibration → reward/continuation model → planning under constraints → actor/critic imagination → F1 25 online data → cross-domain fine-tuning. Each arrow needs evidence.

Check your understanding before continuing

Why must the UI use the training normalizer?

What should determine the displayed simulation horizon?

When is a new architecture justified?

Solutions and reasoning

1. The model learned parameters in that transformed feature space; a different transform changes the meaning of every input/output. 2. Matched held-out horizon error, physical guardrails, uncertainty and operational tolerance—not the configured output length. 3. When a specific measured failure survives data/contract fixes, objective improvements and simpler baselines, and the proposed mechanism directly targets that failure.

Independent build challenge

Rebuild APEX from an empty repository using only the canonical contract, acceptance tests and architecture diagram. Then propose one original model change with a falsifiable hypothesis, matched ablation and rollback condition.

Do this without copying the solution. Your goal is not identical code; your goal is an implementation whose assumptions you can explain and test.

Ten-project ladder

The small labs teach one idea in isolation. The projects force multiple ideas to coexist.

One-dimensional car integrator

Telemetry quality laboratory

Lap physics sandbox

Nonlinear transition model

Track segment encoder

GRU telemetry forecaster

Selective SSM memory laboratory

RSSM imagination laboratory

Latent MPC planner

Project APEX production F1 world-simulation engine

For every project, write an Architecture Decision Record before opening the supplied solution. Record the problem, options, evidence, decision, and revisit condition.

Primary references

Hafner et al., *Mastering Diverse Domains through World Models (DreamerV3)*, arXiv:2301.04104.

Gu and Dao, *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*, arXiv:2312.00752.

Assran et al., *Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture*, arXiv:2301.08243.

LeWorldModel authors, project paper and official implementation.

FastF1 official documentation.

OpenF1 official documentation.

Electronic Arts, *F1 25 Data Output Specification*.

PyTorch official documentation for Dataset, DataLoader, GRU, autograd and optimization.